香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

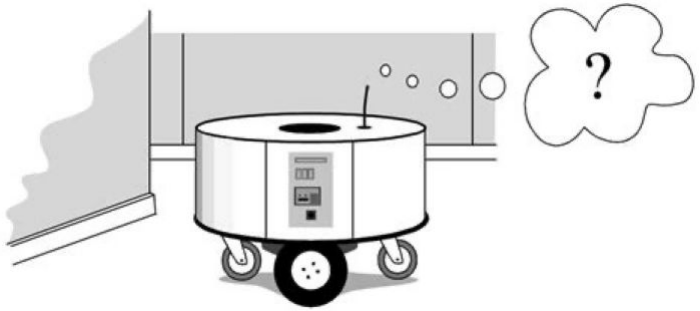
Embodied AI

Lecture 9 & 10: Classic Robotics: Planning and Control

Junwei Liang
AI Thrust
HKUST (Guangzhou)

Agenda

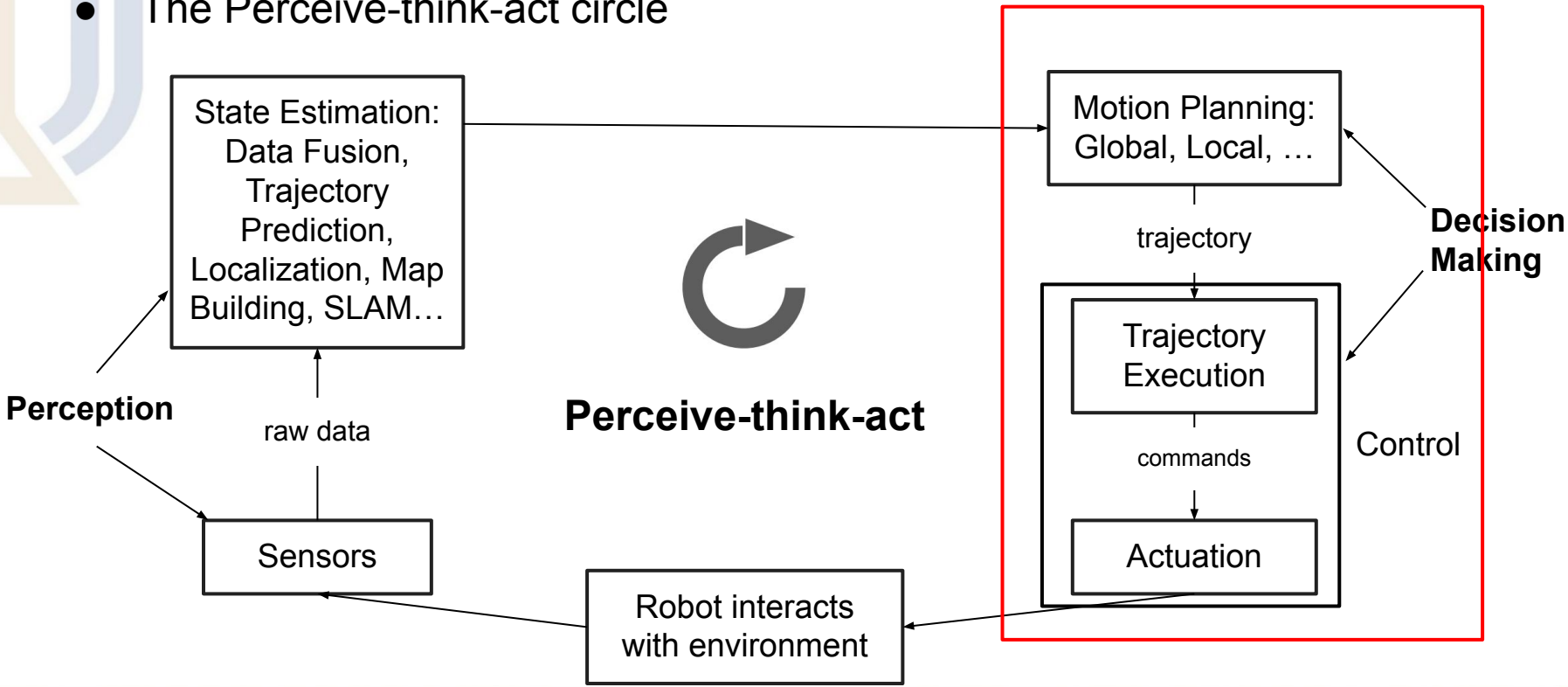
- Objectives
  - We have introduced classic state estimation methods. Now we will take a look at key classic robotics topics of planning and control. We will have Quiz test 2 and paper presentation #1.
- The tasks
  - Motion Planning
  - Feedback Control
  - Quiz 2
  - Paper presentation #1



* Slide credit: Chris Mavrogiannis; Jeannette Bohg; Cyrill Stachniss; Hao Su

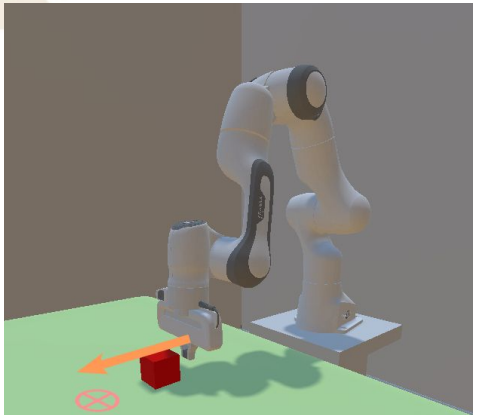
Recap - The Embodied AI Paradigm

- The Perceive-think-act circle

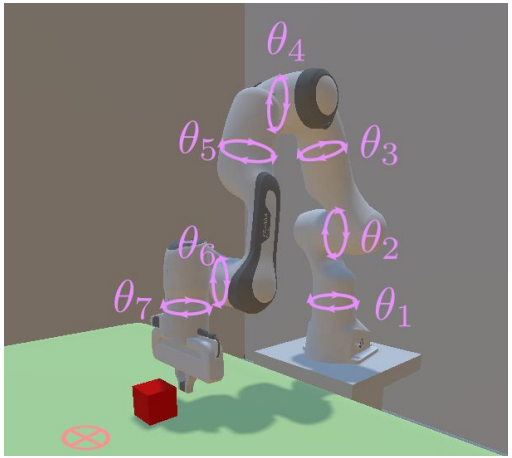


Introduction: How to move a robot arm from pose A to B?

- For example, a table-top Franka robot arm with 7 degree-of-freedom
  - This means we need some policy to map observations to **actions of 7 joints of the robot arm**

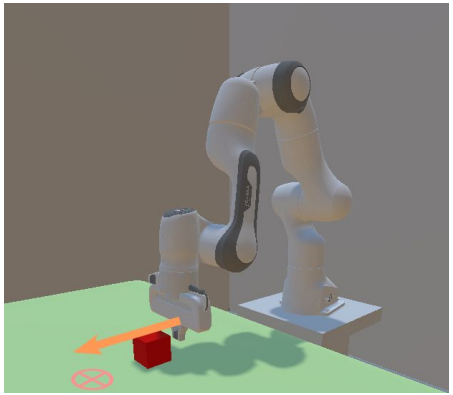


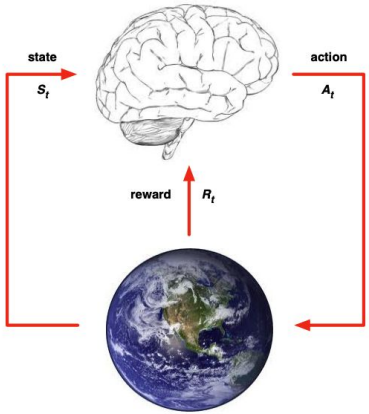
?

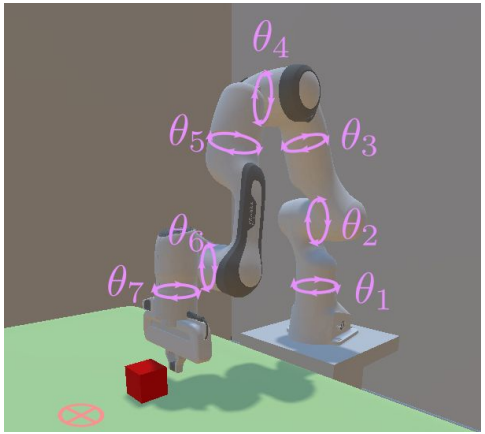


Introduction: How to move a robot arm from pose A to B?

- Using Reinforcement Learning, we can set the reward function (e.g., the red block distance to goal location) and use PPO, letting the robot/policy to figure out in hundreds of thousands of tries/rollouts

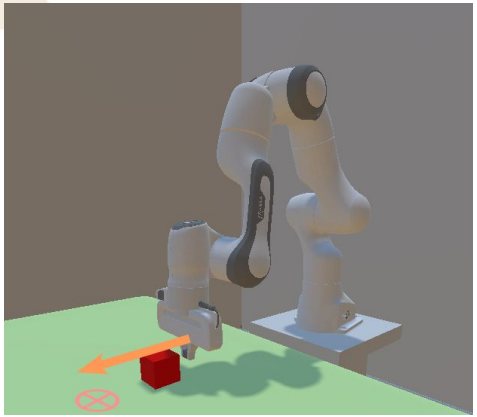




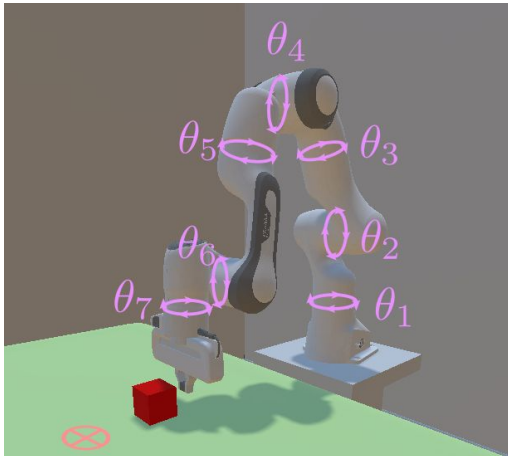


Introduction: How to move a robot arm from pose A to B?

- But before reinforcement learning, when computation is limited, how do classic robotics solve this?

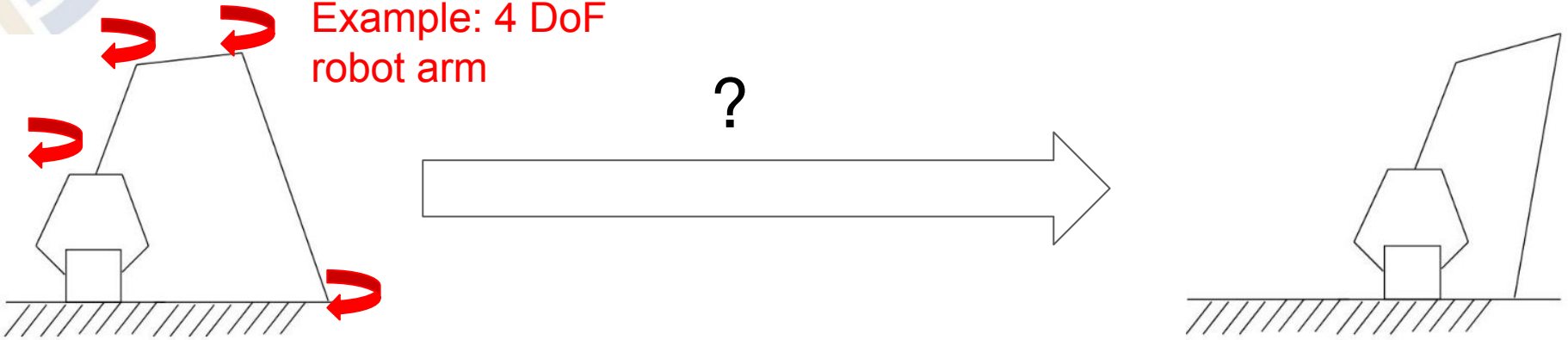


?



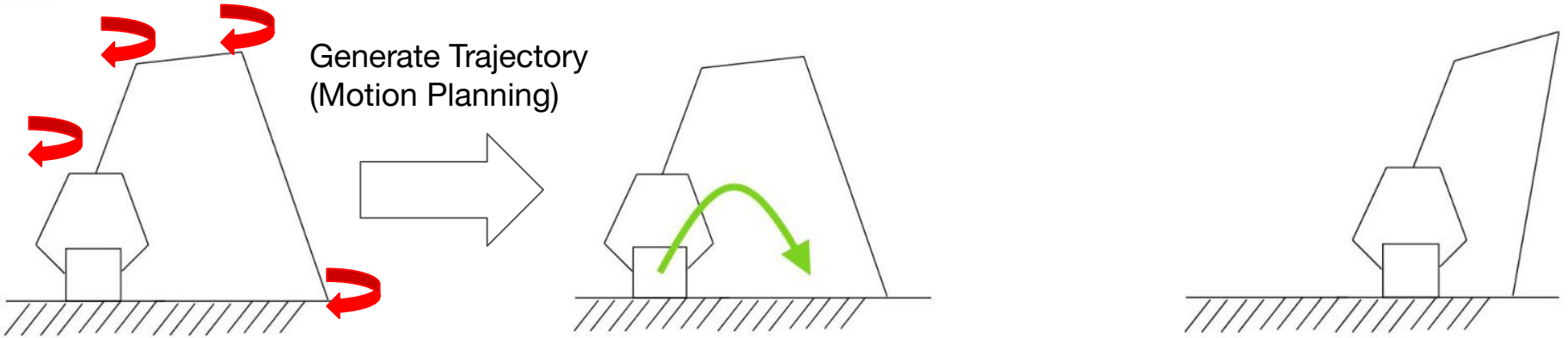
Introduction: How to move a robot arm from pose A to B?

- First, they use simplified physics models
- And then use the **motion planning** and **control** pipeline



Introduction: How to move a robot arm from pose A to B?

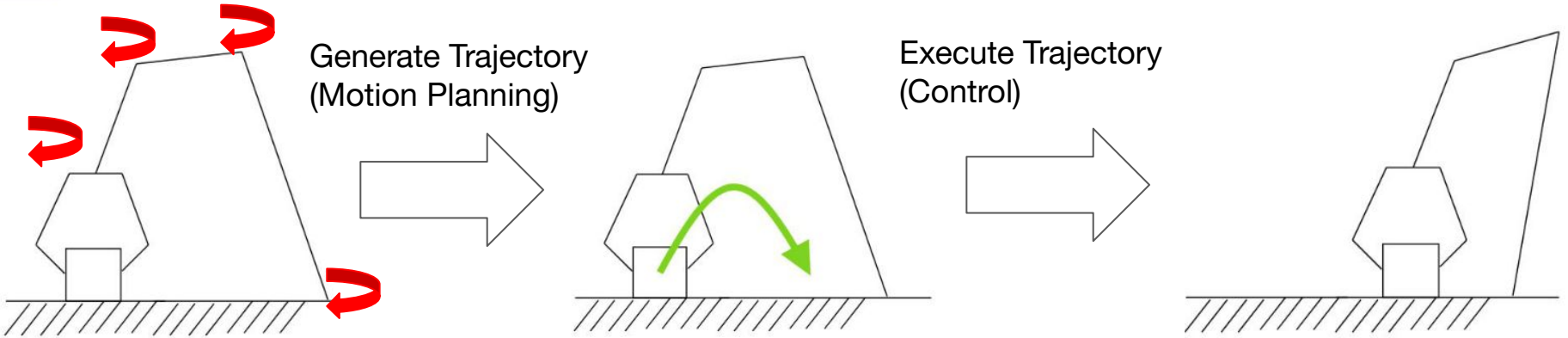
- Use the **motion planning** and **control** pipeline
- Motion planning generates a trajectory (position, velocity, and acceleration) of the robot.



Example: 4 DoF robot arm

Introduction: How to move a robot arm from pose A to B?

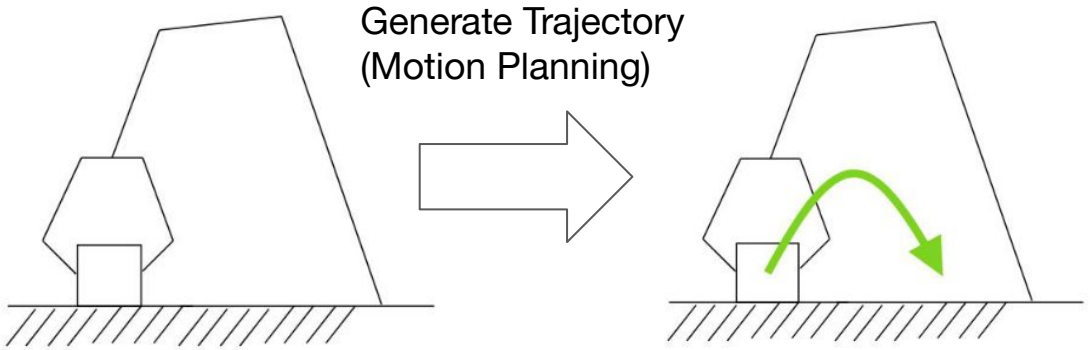
- Use the **motion planning** and **control** pipeline
- Motion planning generates a trajectory (position, velocity, and acceleration) of the robot.
- Control executes the trajectory.



Example: 4 DoF robot arm

Let’s Start with Motion Planning

- Motion planning generates a trajectory (position, velocity, and acceleration) of the robot.



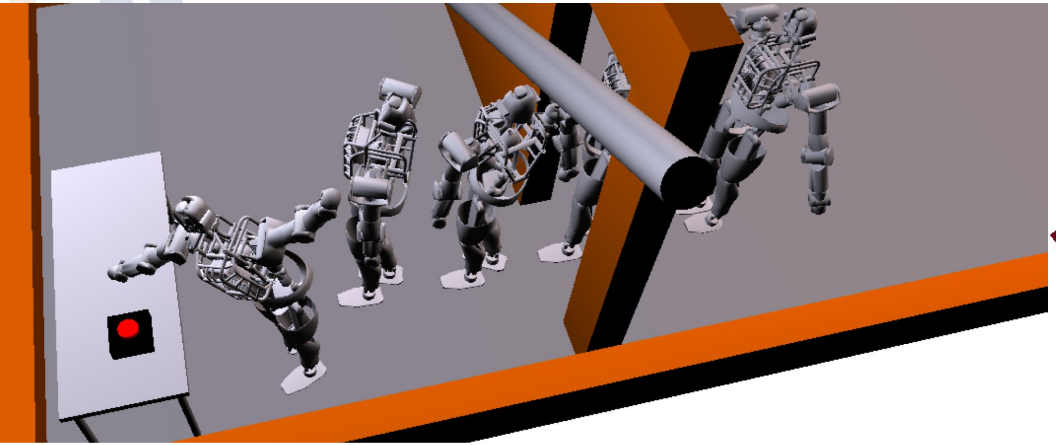
Manipulation Planning Example

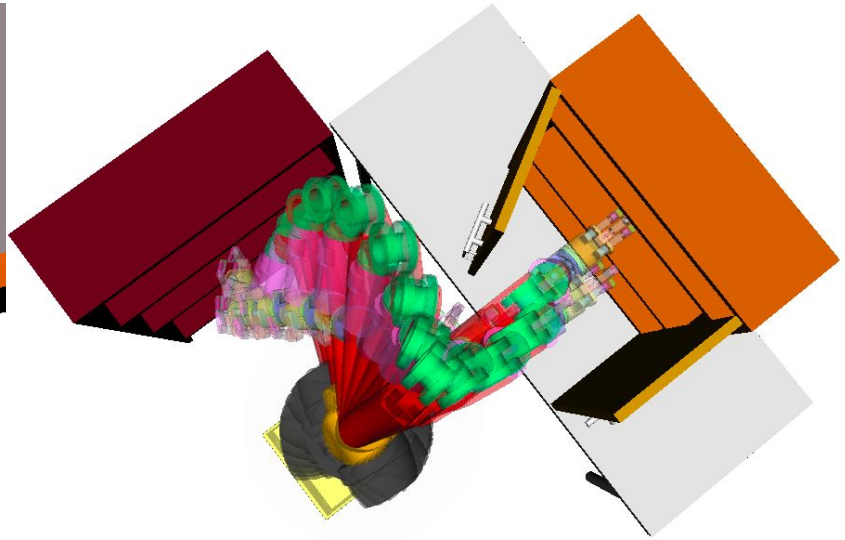
- Task: move a robot from one pose to another
  - Robot moving objects on the table (from 2012)



Manipulation Planning Example

- Task: move a robot from one pose to another



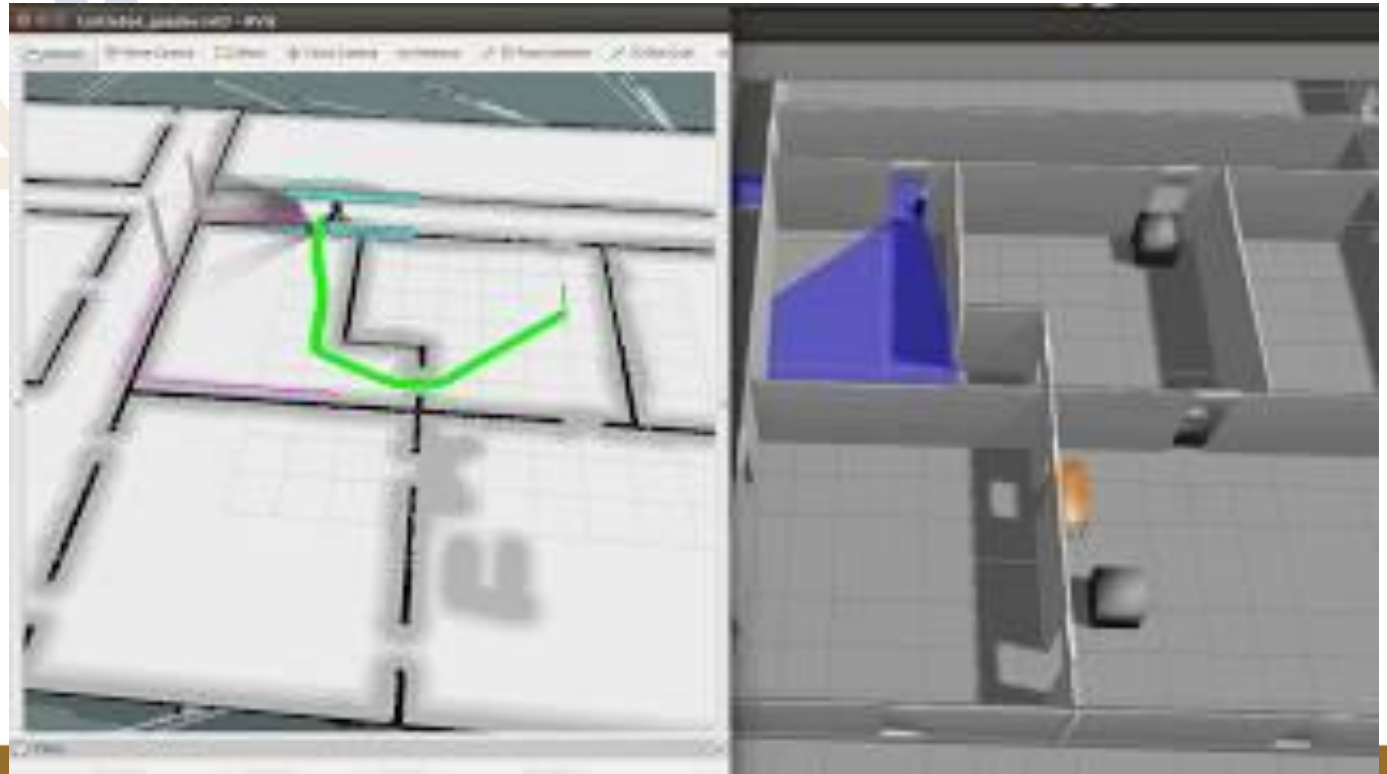


Ratliff N, Zucker M, Bagnell J A, et al. CHOMP: Gradient optimization techniques for efficient motion planning, ICRA 2009
Schulman, John, et al. Finding Locally Optimal, Collision-Free Trajectories with Sequential Convex Optimization, RSS 2013

https://www.youtube.com/watch?v=U9BJkkJ0nOc

Navigation Planning Example

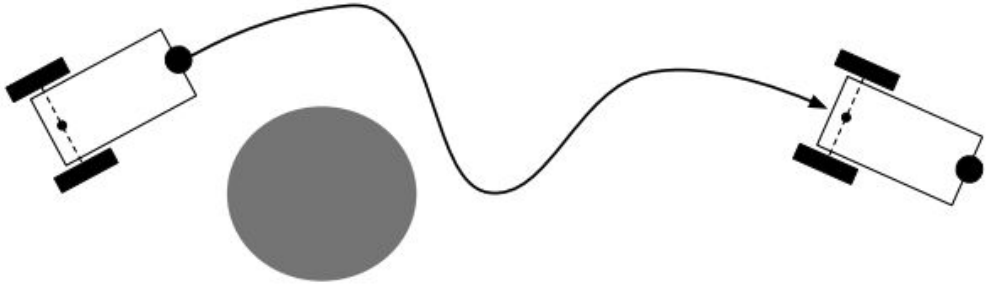
- You can build a turtlebot to navigate in Gazebo (from ROS) simulator



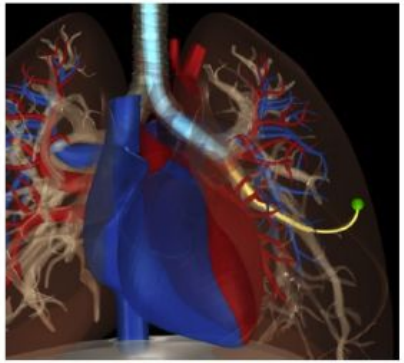
The simulation is run in real-time with a navigation module. You can add obstacle in real-time and the navigation will change its plan, after it has visually detected the obstacle.

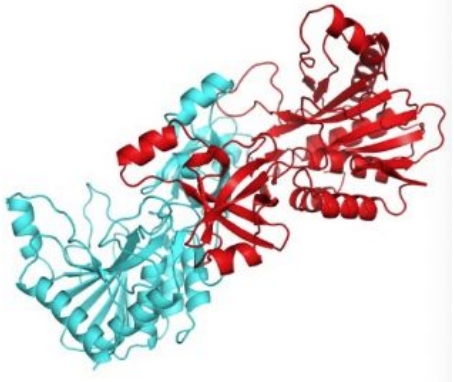
More examples of motion planning

- Steering autonomous vehicles
- Controlling humanoid robot
- Surgery planning
- Protein folding
- …









Some history

- Planning is formally defined in the 1970s
- Development of exact, combinatorial solutions in the 1980s
- Development of sampling-based methods in the 1990s
- Deployment on real-time systems in the 2000s

Planning - Problem Formulation

Given:

1. An initial state (pose) of the world
2. A set of available actions, their requirements, and their effects
3. A goal state (pose)

Roboticists use planning for navigation problems so pose is often mentioned

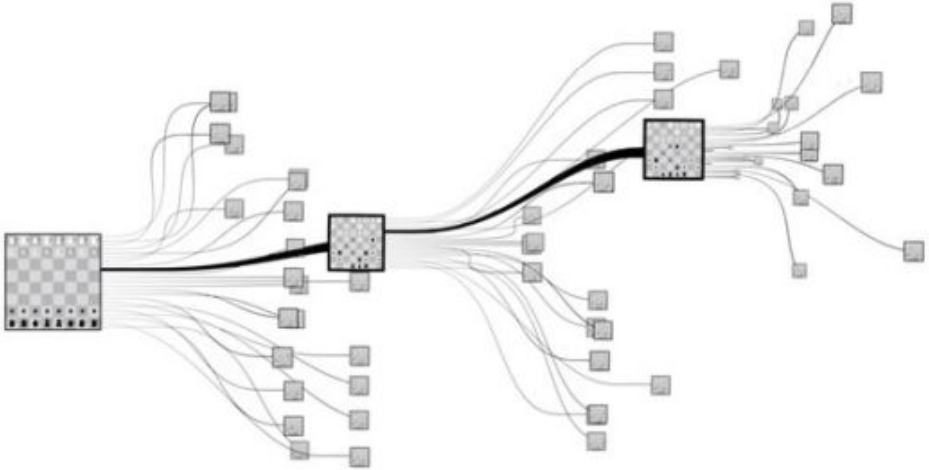
Compute:

A valid sequence of actions that starts from **the initial state** and terminates at **the goal state**

~1970-1990s

- Games
  - **Can we apply planning to play chess?**





Planning For Chess

Given:

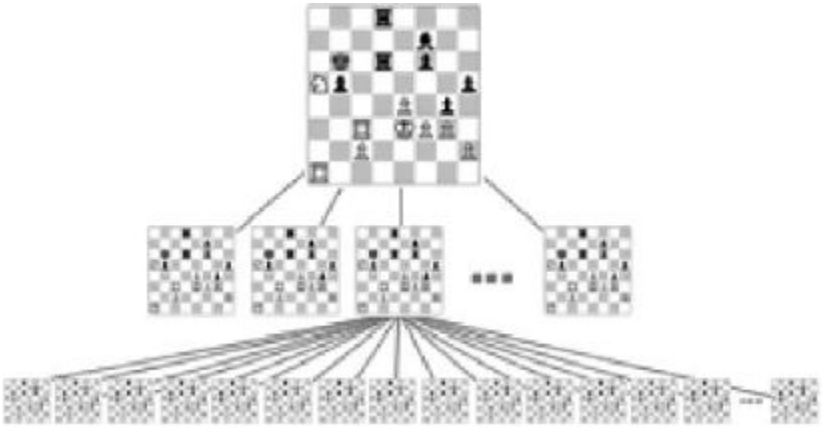
1. An initial state of the world **=> The chess board state at each turn**
2. A set of available actions, their requirements, and their effects **=> Chess moves**
3. A goal state **=> Checkmate conditions**

Compute:

A valid sequence of actions that starts from the initial state and terminates at the goal state => **Plan = sequence of chess moves**

Planning Via Tree Search

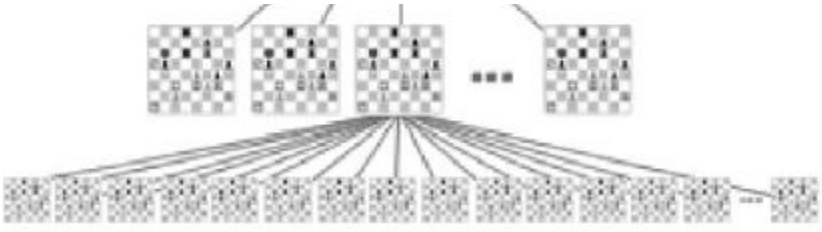
1. Enumerate all valid moves available to player
2. Enumerate all valid moves available to opponent
3. Repeat 1, 2 until the game ends
4. Pick action that had the highest likelihood of winning



Planning Via Tree Search

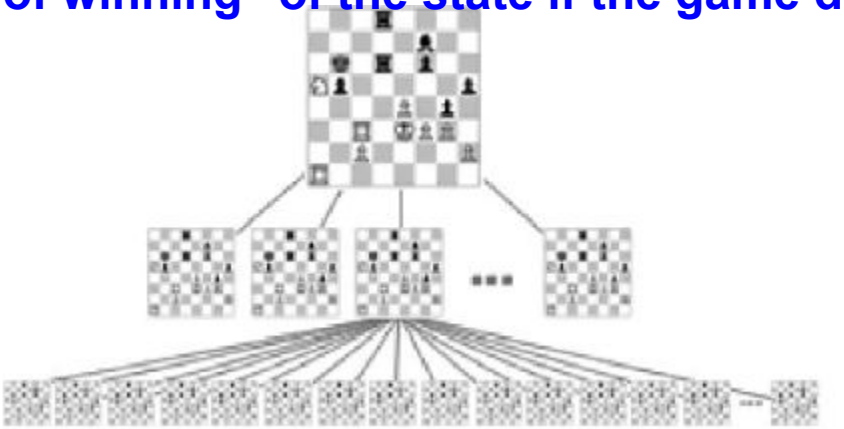
1. Enumerate all valid moves available to player
2. Enumerate all valid moves available to opponent
3. Repeat 1, 2 until the game ends
4. Pick action that had the highest likelihood of winning

Problem: Steps 1, 2 will enumerate far more states than computationally tractable!



Planning Via Tree Search

1. Enumerate all valid moves available to player
2. Enumerate all valid moves available to opponent
3. Repeat 1, 2 until the game ends **or up to d steps ahead**
4. Pick action that had the highest likelihood of winning, **using an estimated "value/chance of winning" of the state if the game did not end.**



1997: IBM’s Deep Blue defeats Garry Kasparov

Algorithm: Tree search with some clever strategies to prune the tree (alpha-beta pruning)

"Secret Sauce": Special chips designed to speed up enumeration and evaluation of chess moves



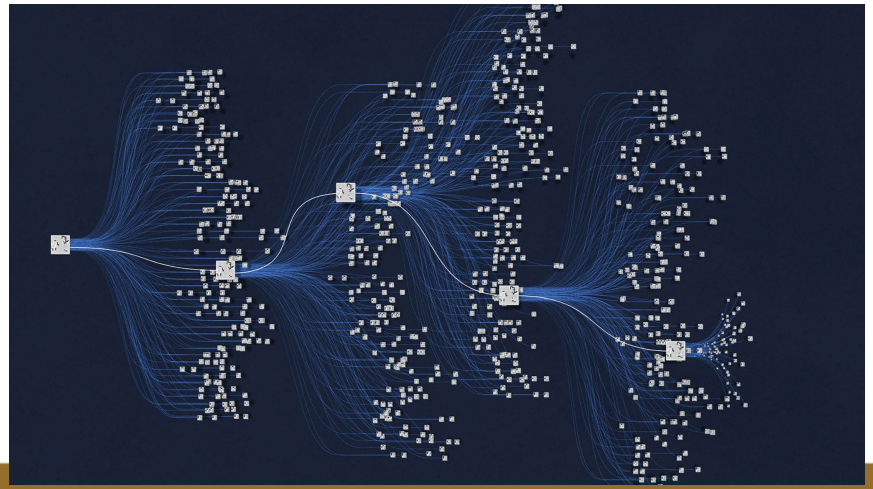
Limitations of Playing Games Via Tree Search

1. Enumerate all valid moves available to player
2. Enumerate all valid moves available to opponent
3. Repeat 1, 2 until the game ends or up to d steps ahead
4. Pick action that had the highest likelihood of winning, using an estimated "value" of the state if the game did not end.

There are games harder than chess (e.g. Go) where steps 1,2 cannot be done even for a small number of steps of lookahead!

AlphaGo: Tree Search + Policy + Value Neural Networks

1. Enumerate ~~all~~ **promising moves from policy neural network**
2. Enumerate ~~all~~ **promising moves** available to opponent (same network)
3. Repeat 1, 2 until the game ends **or up to d steps ahead**
4. Pick action that had the highest likelihood of winning, **using learned value neural network**



From Games to Robotics

- Discrete state space to continuous state action space
- Easy to simulate moves - no expensive physics / geometric computation
  - Simply place the piece on the chess board
- Rules of game already known - no notion of model uncertainty
  - We can compute the value/reward easily based on the game rules
  - In robotics, we need to simulate the robot or run the program on real robot to learn what will happen

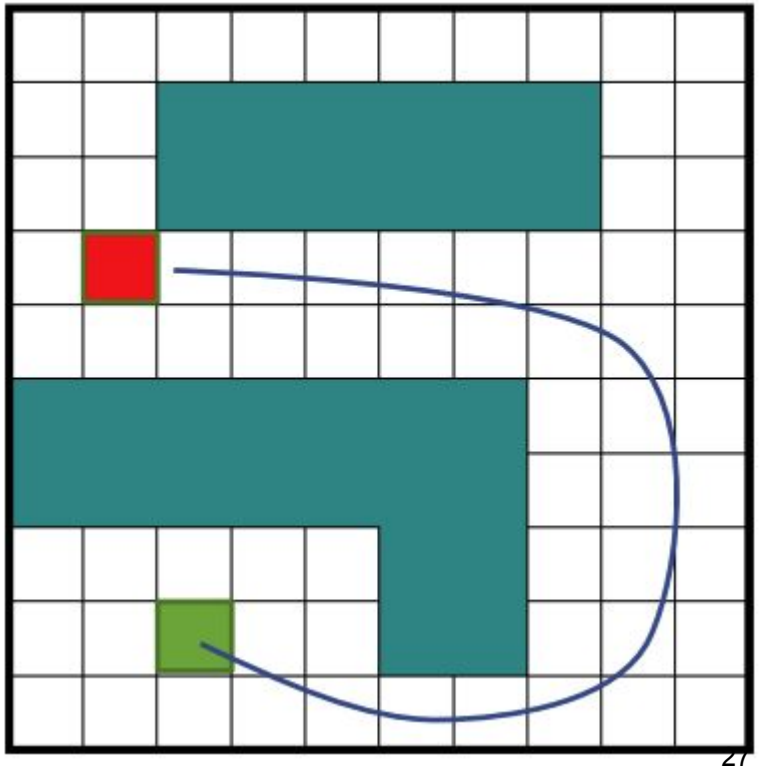
Popular Approaches

- **Potential fields** *[Rimon, Koditschek, '92]*: create forces on the robot that pull it toward the goal and push it away from obstacles
- **Grid-based planning** *[Stentz, '94]*: discretizes problem into grid and runs a graph-search algorithm (Dijkstra, A*, …)
- **Combinatorial planning** *[LaValle, '06]*: constructs structures in the configuration space (C-space) that completely capture all information needed for planning
- **Sampling-based planning** *[Kavraki et al, '96; LaValle, Kuffner, '06, etc.]*: uses collision detection algorithms to probe and incrementally search the C-space for a solution

We will cover these

Grid-based Approaches

- Discretize the continuous world into a grid
  - Each grid cell is either free or forbidden
    - a.k.a. Occupancy grid map
  - Robot moves between adjacent free cells
  - **Goal:** find sequence of free cells from start to goal
- Mathematically, this corresponds to pathfinding in a discrete **graph** G = (V, E)
  - Each vertex v represents a free cell
  - Edges connect adjacent grid cells
  - So let's look into graph search algorithm

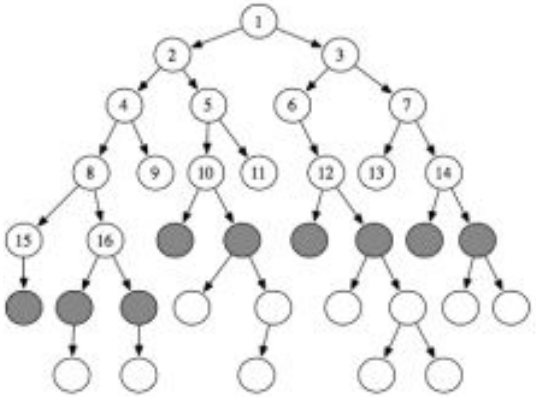


Graph Search Algorithms

- Having determined problem, how to find “best” path?
  - Breadth-first search

Breadth-First Search

- Maintain the path as a **queue** – First in/first first out
- Update cost for all edges up to current depth before proceeding to greater depth
- Edges may have different cost
- Vertices explored in order of distance from start
- Best path found = path with fewest steps



Dijkstra's Algorithm: BFS + Priority

- Accounting for edge costs
- Changes queue to priority queue
  - So the path with the shortest distance is always at the front
- Edit previous entry only if new option is better!

You only need to know the intuition behind this algorithm through visualization in the next slides

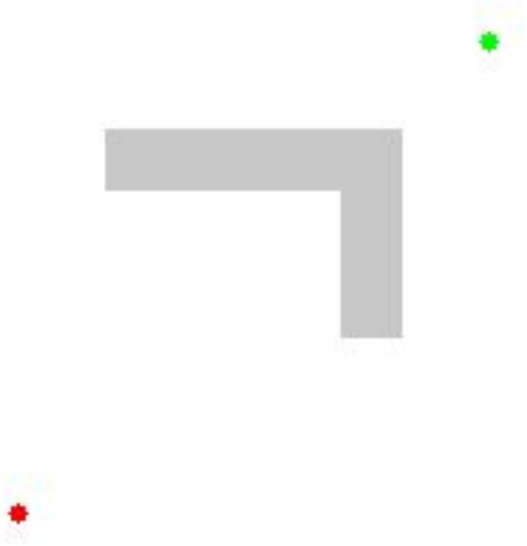
```
frontier = PriorityQueue()  // Note priority!
frontier.push(start, 0)     // 0 = cost to go
parent = {}, parent[start] = Null
cost = {}
cost[start] = 0             // Cost from start

while not frontier.empty():
  current = frontier.get()  // Get by priority!
  if current == goal:
    break
  for next in neighbors(current):
    new_cost = cost[current] + EdgeCost[current, next]
    if next not in cost or new_cost < cost[next]:
      cost[next] = new_cost  // Insertion or edit
      frontier.put(next, new_cost)
      parent[next] = current
```

Dijkstra’s Algorithm: BFS + Priority

You only need to know the intuition behind this algorithm

- **Visualization**
  - Start from bottom left node to a goal node; set all other nodes to be unvisited with infinite cost
  - color representing distant to goal (greener = closer)



A visited node will not be checked again;
Stop when destination node has been marked visited

Dijkstra’s Algorithm: BFS + Priority

- **Visualization**
  - Start from bottom left node to a goal node; set all other nodes to be unvisited with infinite cost
  - color representing distant to goal (greener = closer)



A*: Improving Dijkstra

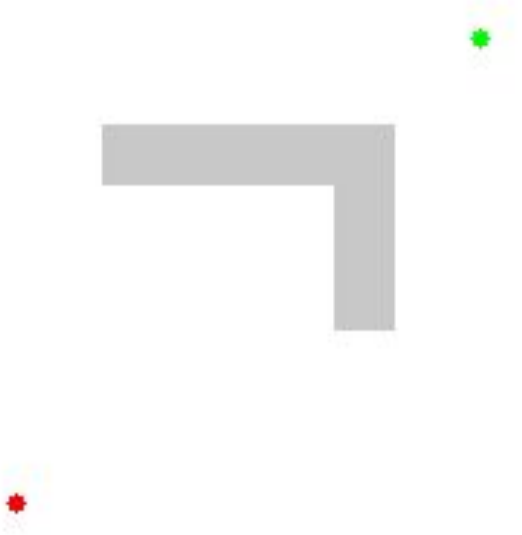
- Dijkstra orders the path queue by optimal “cost-to-arrival”
- We can get faster results if order by “cost-to-arrival”+ (approximate) “cost-to-go”
  - So not just computing the cost of each node to the goal node but also a heuristic for optimal cost-to-go to each node
- In this way, fewer nodes will be placed in the frontier queue
- This modification still **guarantees** that the algorithm will terminate with a shortest path

$$f(n) = \underbrace{g(n)}_{\text{cost from start to } n} + \underbrace{h(n)}_{\text{estimated remaining cost to goal}}$$

Setting h=0 gives Dijkstra

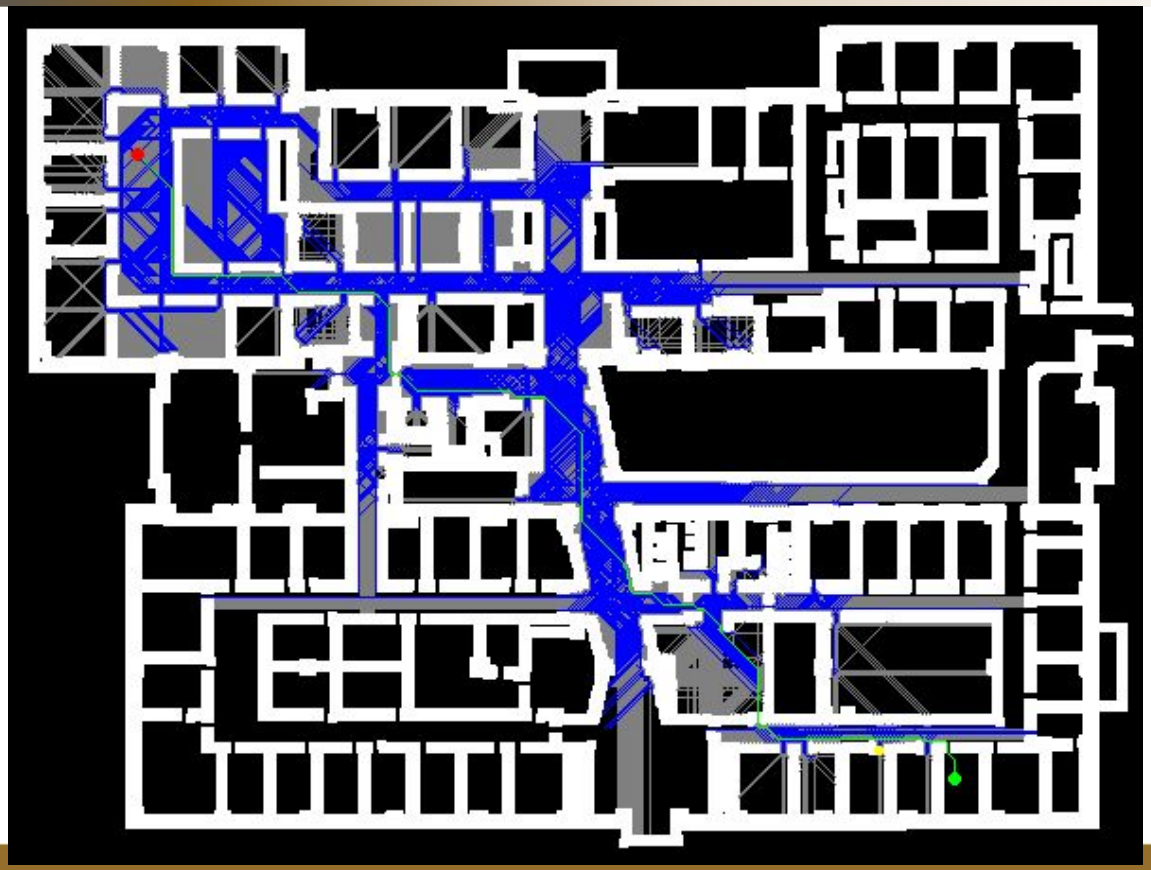
A*: Improving Dijkstra

- Visualization



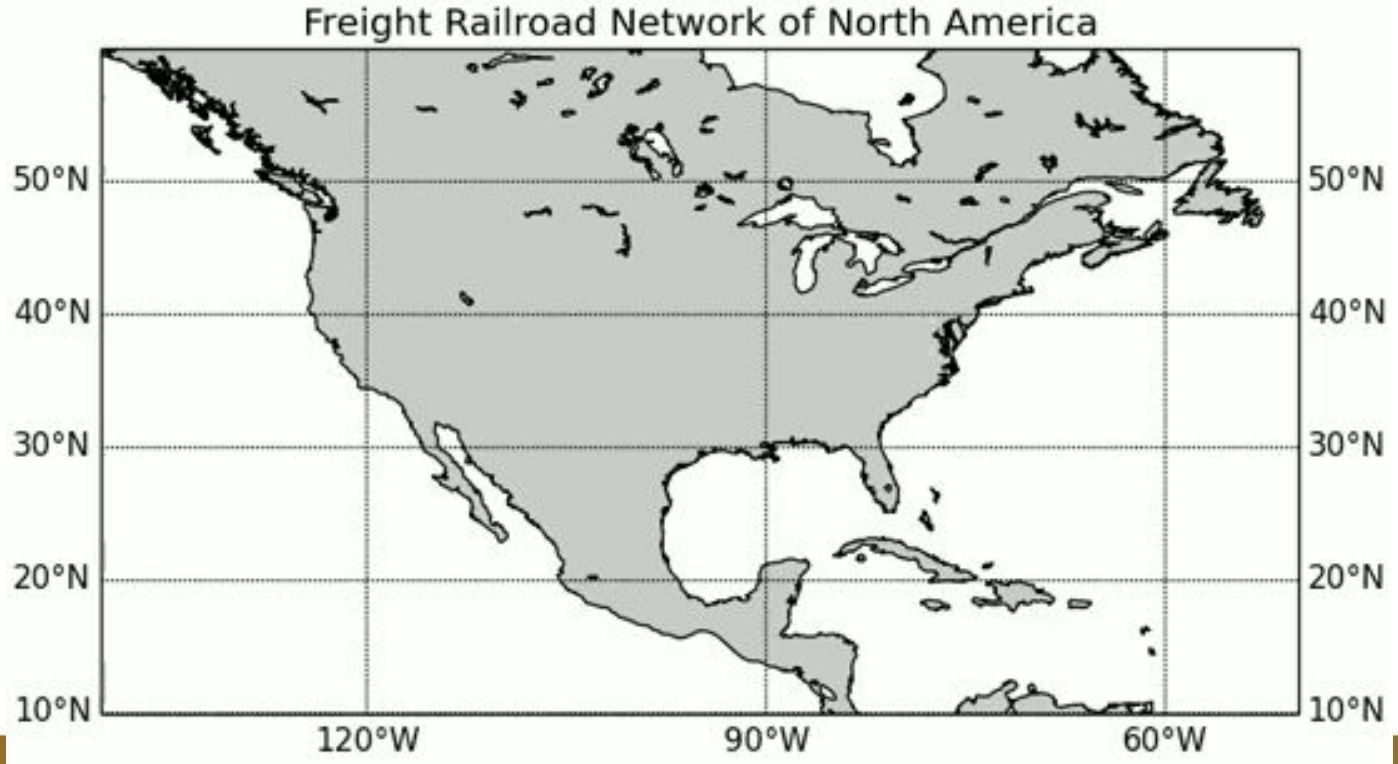
Grid Planning For Real Robots, In The Real World

Given the occupancy grid map, start and end point. The blue are the traversed nodes and the green line is the optimal path



A* to search a route between D.C. and LA

- Real-world application: finding an optimal path in the railroad network



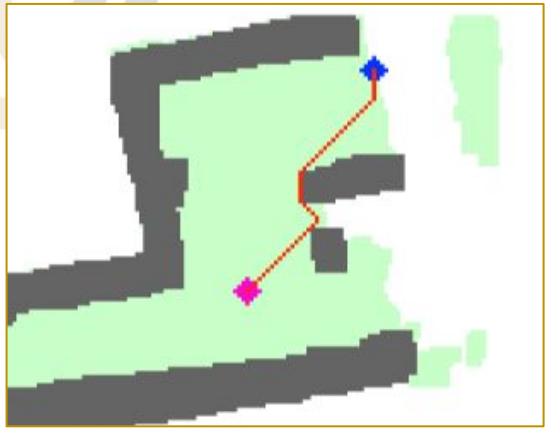
Grid-based approaches: summary

- Pros:
  - Simple and easy to use
  - Fast (for some problems)
- Cons
  - Limited to simple robots
    - Grid size is exponential in the number of DOFs
- Good tutorials for graph-search algorithm
  - https://github.com/npretto/pathfinding
  - For A*: https://www.redblobgames.com/pathfinding/a-star/introduction.html

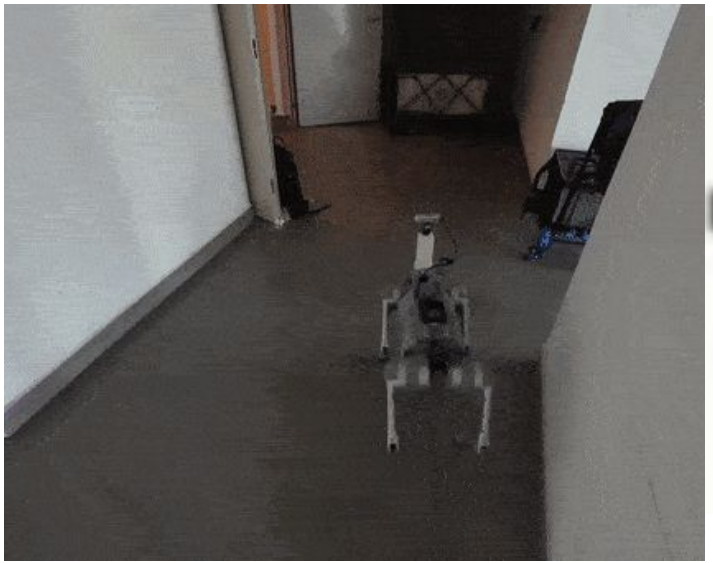
In practise, A* is used extensively in many problems including indoor navigation with a known 2D map.

A* for Navigation

- In practise, A* is used extensively in many problems including indoor navigation with a known 2D map. See our robot navigation demo with Go2



Using VLM and depth camera we can construct an occupancy+frontier map like this. We need motion planning A* to get a trajectory.



Find the elevator lobby



To continuous motion planning

- A robot is a geometric entity operating in continuous space
  - In navigation we might cheat a little using A* in discrete occupancy map (consider the robot as a point)
  - But for manipulation, A* does not work since we need fine-grained motion trajectories
- Task: move a robot from one pose to another
- Assumptions
  - We know the start and goal pose
  - **We can verify if a random pose is valid (usually means collision-free)**
  - We can verify whether a pose is reachable from another pose using some simple control strategy
- Algorithms (Sampling-based)
  - **Probabilistic Roadmap (PRM)**
  - **Rapidly-exploring Random Trees (RRT)**

Sampling-based Motion Planning

- Pros:
  - Conceptually simple
  - Relatively-easy to implement
  - Flexible: one algorithm applies to a variety of robots and problems
  - Beyond the geometric case: can cope with complex differential constraints, uncertainty, etc.
- (Mild) cons:
  - Unclear how many samples should be generated to retrieve a solution
  - Cannot determine whether a solution does not exist
- Two major approaches
  - **Probabilistic Roadmap (PRM): graph-based**
  - **Rapidly-exploring Random Trees (RRT): tree-based**

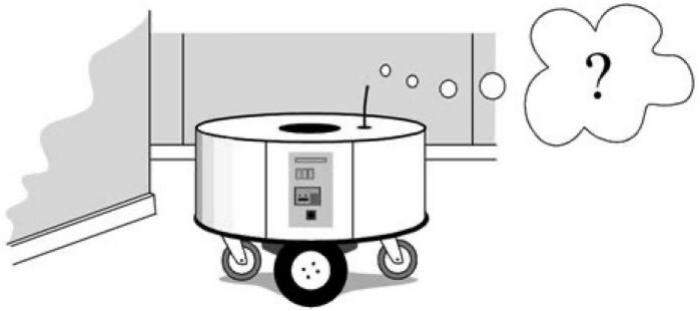
A* is better than RRT in some cases[1]

Due to time limit, we won't go into details about sampling-based planning methods. You can check out the backup slides if you are interested.

[1] Zammit, Christian, and Erik-Jan Van Kampen. "Comparison between A* and RRT algorithms for 3D UAV path planning." Unmanned Systems 10.02 (2022): 129-146.

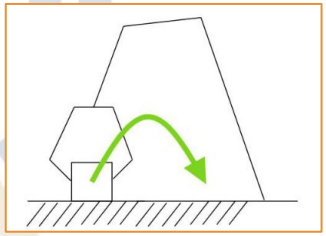
Agenda

- Objectives
  - We have introduced classic state estimation methods. Now we will take a look at key classic robotics topics of planning and control. We will have Quiz test 2 and paper presentation #1.
- The tasks
  - Motion Planning
  - **Feedback Control**
  - Quiz 2
  - Paper presentation #1

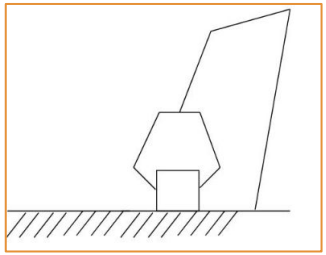


* Slide credit: Chris Mavrogiannis; Jeannette Bohg; Cyrill Stachniss; Hao Su

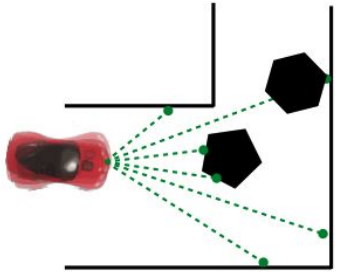
Control



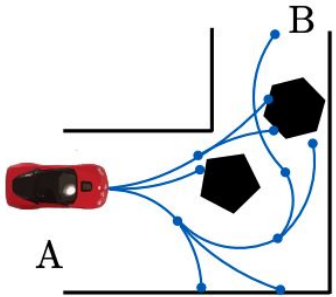
Execute
Trajectory
(Control)



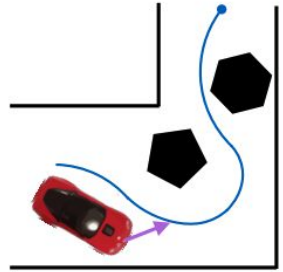
Estimate state



Plan a sequence of motions



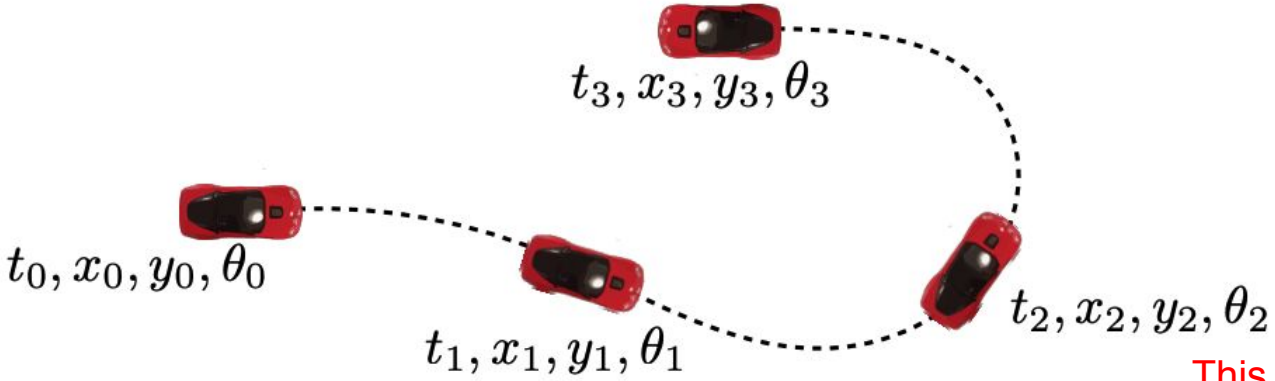
Control robot to follow path



- Robot pose known
- Path is given

The control framework

- Say we want to get the car to reach a sequence of poses



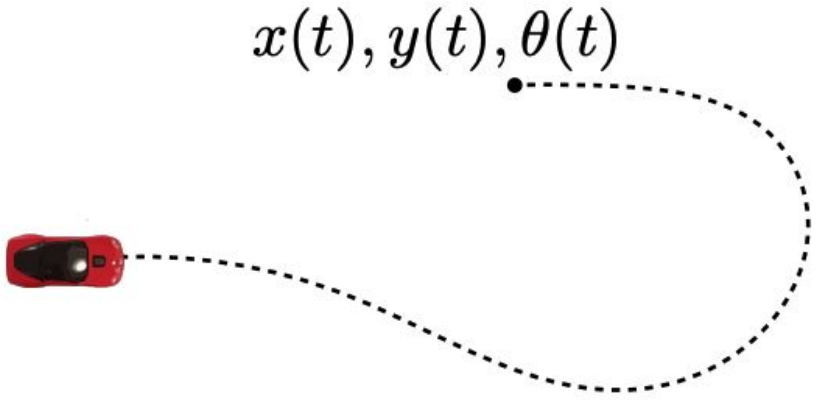
Can express this as wanting to track a reference *trajectory*

$$x(t), y(t), \theta(t)$$

This example is for a planar robot with poses defined in SE(2). The trajectory comes from the planning module

The control framework

- Let's say we want to track a **reference trajectory**



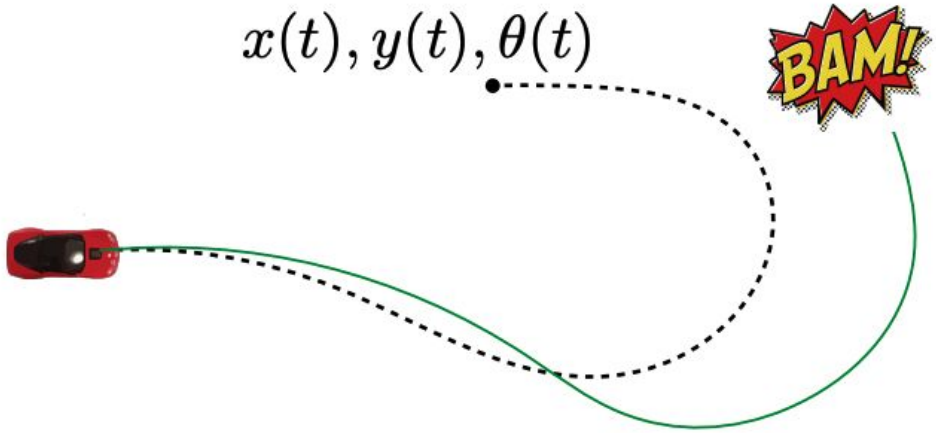
Given time step t, we have a reference pose;
We need to convert that to the control commands, like the steering angle and acceleration

Objective: Figure out a **control** trajectory $u(t)$ to achieve this

In our case, we will focus on steering angle $\delta(t)$ as control input

Why do we need feedback?

- Let's say we want to track a reference trajectory



Control in the real world is **never** perfect! Say you turn 1 degree, the car will never turn exactly 1 degree

What if we send out steering angles $\delta(t)$ obtained from kinematic car model?

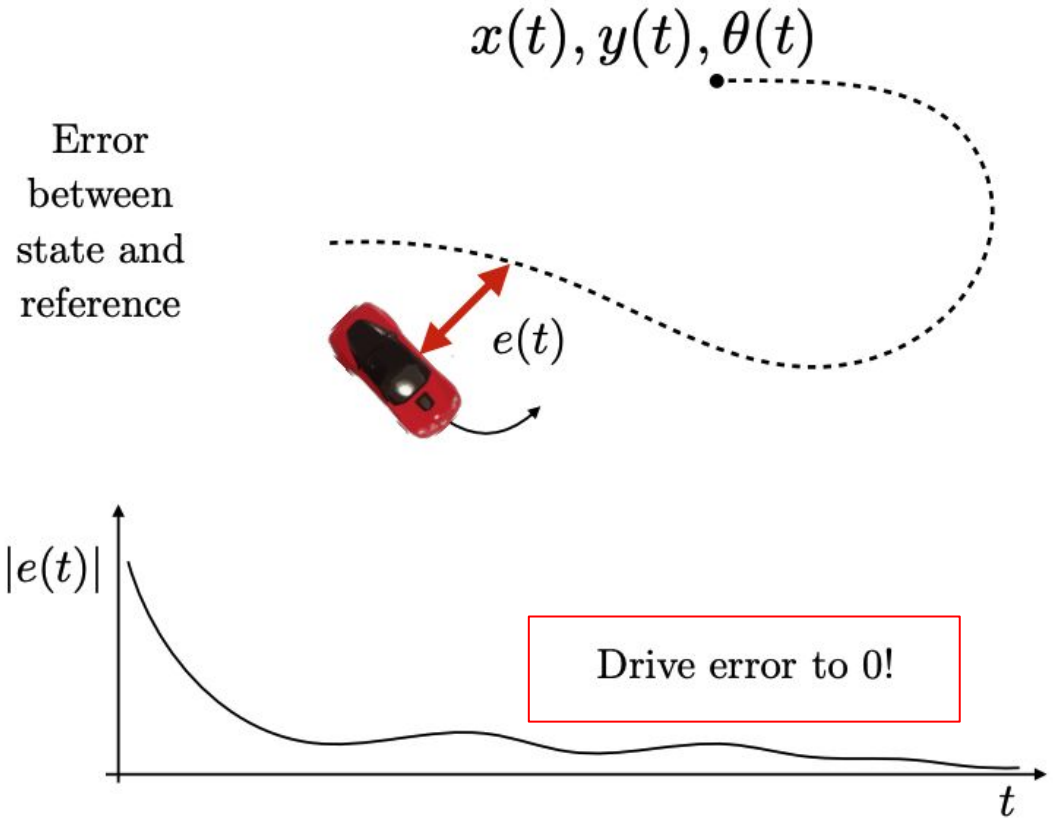
This **open loop control** will lead to large errors.

Open loop control: the controller is not affected by the state or any **feedback**.

Why do we need feedback?

- Principle of feedback control:
  - **Measure error between** reference and state at all times
  - Minimize this error

Measure error and minimize it

- 

Is this still a research problem?

- Industrial robots at work



These robots are usually assumed “fully actuated” and there is little environmental disturbance. Feedback control is often not needed.

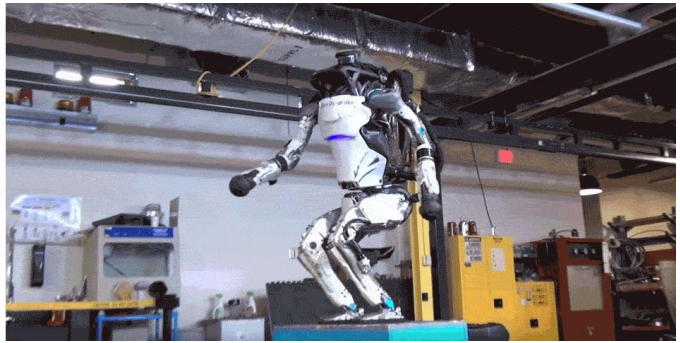
Assumptions made by such controllers

- 1. **Fully actuated**: There exists an inverse mapping from reference to control actions
- 2. Almost no execution error or state estimation error
- 3. Enough control authority to clamp down errors / overcome disturbances
  - No external influence factors
  - In factory, the robots can be re-calibrated over a time period

Instead, what are harder problems for control?

- In real-world, there are many underactuated systems
- What affects the error between robot state and reference?



Some initial motor thrust ...

Whole lot of gravity!

Whole lot of momentum!

... some precise control adjustments

Instead, what are harder problems for control?

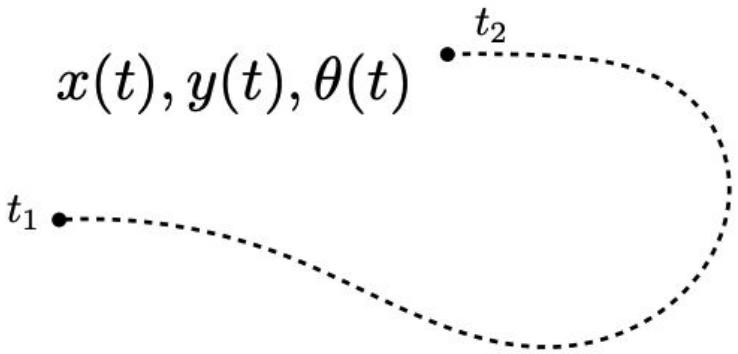
- Other challenges for mobile robot control
  - 1. Unexpected obstacle avoidance
  - 2. Noisy state estimation
  - 3. Motion model changing on the fly (drifting race cars, etc.)

A generic template for a controller

- 1. Get a reference path / trajectory to track
- 2. Pick a point on the reference
- 3. Compute error to reference point
- 4. Compute control law to minimize error

Step 1: Get a reference

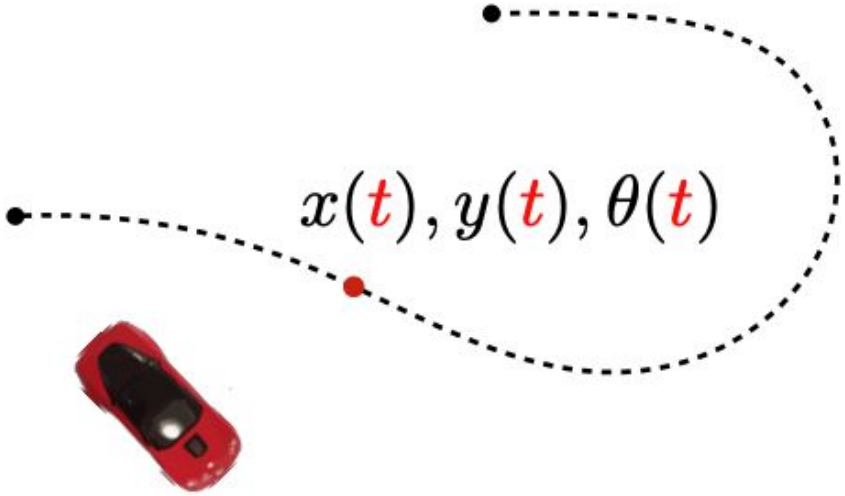
- Reference can be a time-parameterized trajectory



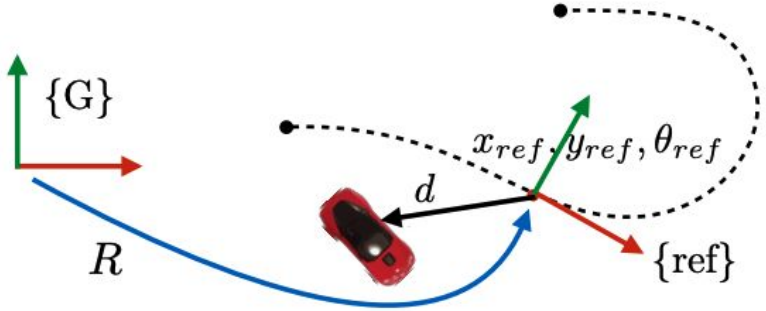
The time index refers to a desired pose we want the system to achieve at a given time

Step 2: Pick a point on the reference

- Pick a reference point on the trajectory / path



Step 3: Compute error to reference point



1. Define error vector

$$d = \begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} x_{ref} \\ y_{ref} \end{bmatrix}$$

Translation error

2. Rotate error vector to be in the **reference** frame

Rotation Matrix

$$e = R^T(\theta_{ref}) \left(\begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} x_{ref} \\ y_{ref} \end{bmatrix} \right)$$

$$e = \begin{bmatrix} \cos(\theta_{ref}) & \sin(\theta_{ref}) \\ -\sin(\theta_{ref}) & \cos(\theta_{ref}) \end{bmatrix} \left(\begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} x_{ref} \\ y_{ref} \end{bmatrix} \right)$$

Translate the point to the reference point first, then compute the rotation

Step 4: Compute control law

- Compute control action based on instantaneous error

$$u = K(\mathbf{x}, e)$$

control — state — error

(steering angle, speed)

Compute the new control action.
Apply action, robot moves a bit,
Then compute new error, repeat

Different control laws

- **Proportional–integral–derivative (PID) controller**
  - **PD controller is used in many recent embodied AI research**
- Pure-pursuit control
- Linear Quadratic Regulator (LQR)
- **Model Predictive Control (MPC)**

We will cover PID today

Proportional–integral–derivative (PID) controller

- Goal: Select a control law that tries to drive error to zero (and keep it there)



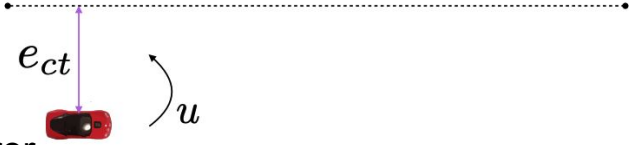
"e_{ct}" refers to "cross-track error": the lateral distance between the robot's current position and the closest point in the reference path

Proportional–integral–derivative (PID) controller

$$u = -\left(K_p e_{ct} + K_i \int e_{ct}(t)dt + K_d \frac{de_{ct}}{dt} \right)$$

Proportional (current) | Integral (past) | Derivative (future)

- Given the error e_{ct}
  - Proportional - get rid of the current error
    - A correction proportional to the error; think position error
  - Integral - if I am accumulating error, try harder
  - Derivative - if I am going to overshoot, slow down
    - Like “**momentum**”; think speed error

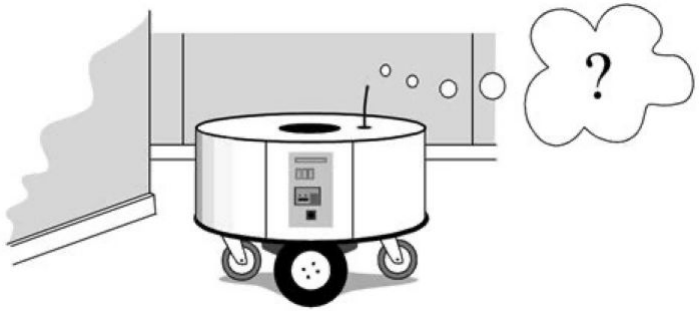


In many recent whole-body-control research, a PD controller is good enough [1][2]

[1] Deep Whole-Body Control: Learning a Unified Policy for Manipulation and Locomotion. CoRL 2022
[2] Visual Whole-Body Control for Legged Loco-Manipulation. ArXiv, 2024

Agenda

- Objectives
  - We have introduced classic state estimation methods. Now we will take a look at key classic robotics topics of planning and control. We will have Quiz test 2 and paper presentation #1.
- The tasks
  - Motion Planning
  - **Feedback Control**
    - **Example for Self-driving Policy (Simulation)**
    - Example for Locomotion Policy (Real Robot)
  - Quiz 2
  - Paper presentation #1



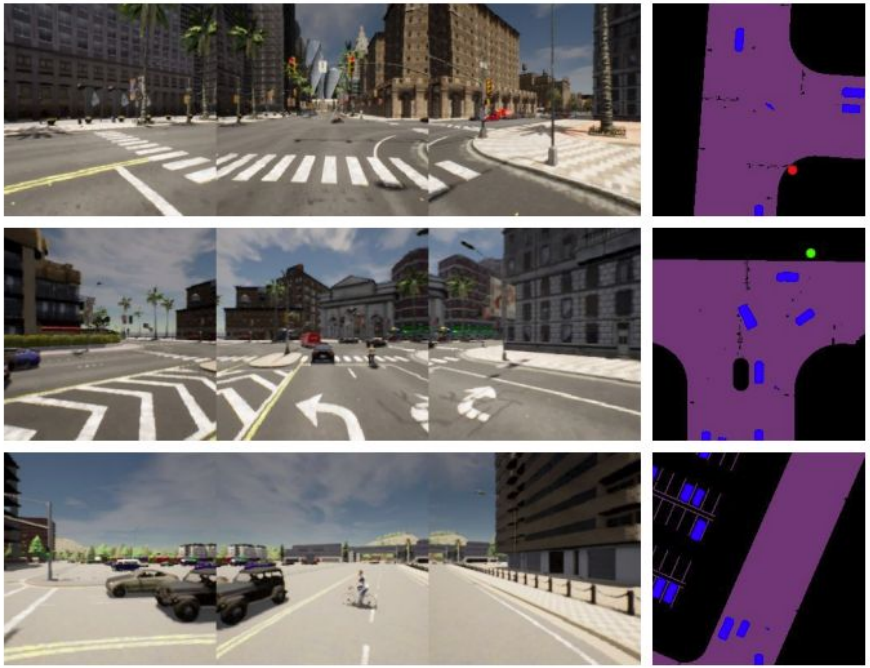
* Slide credit: Chris Mavrogiannis; Jeannette Bohg; Cyrill Stachniss; Hao Su

A recent end-to-end self-driving research work

- NEAT: Neural Attention Fields for End-to-End Autonomous Driving. ICCV'21
  - Let's see how the perception, planning and control fit together

End-to-end autonomous driving with BEV perception*

- Bird's-Eye-View (BEV) Given multiview RGB images,
  - Efficient reasoning about the **semantic, spatial, and temporal structure** is crucial for self-driving
  - Driving happens in **physical 3D space**
  - BEV ≈ 3D space



*NEAT: Neural Attention Fields for End-to-End Autonomous Driving. ICCV'21

End-to-end autonomous driving with BEV perception*

BEV Semantics for Driving

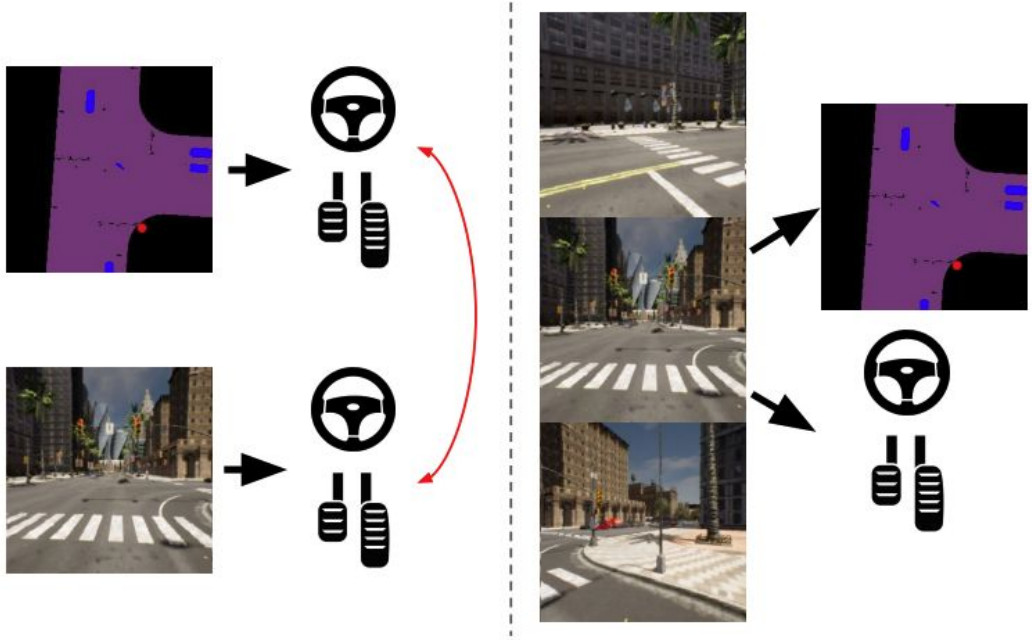
Given multiview RGB images, get BEV representation and driving controls (multi-task learning)

Prior work:

- LBC [CoRL 2019]
- Roach [ICCV 2021]

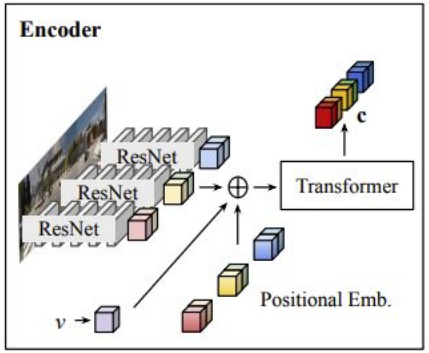
NEAT:

- Multi-task learning
- Single training stage
- Selective attention



End-to-end autonomous driving with BEV perception*

Architecture: Encoder



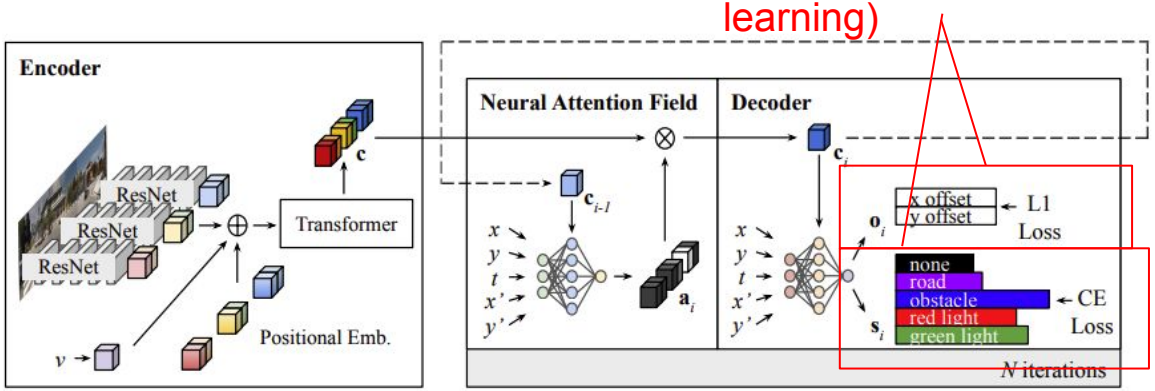
- A **ResNet backbone** yields 64 patches/tokens per image
- Patch features are combined with **velocity** and **positional encoding**
- A **Transformer** integrates contextual cues through self-attention

* NEAT: Neural Attention Fields for End-to-End Autonomous Driving, ICCV 2021

End-to-end autonomous driving with BEV perception*

Architecture: NEAT and Decoder

BEV representation and driving controls (multi-task learning)



The model is trained through imitation learning, where they have collected human driver demonstration paths with waypoints.

Waypoint: position of the vehicle (in the vehicle's local coordinate system) in the expert demonstration at time-step t

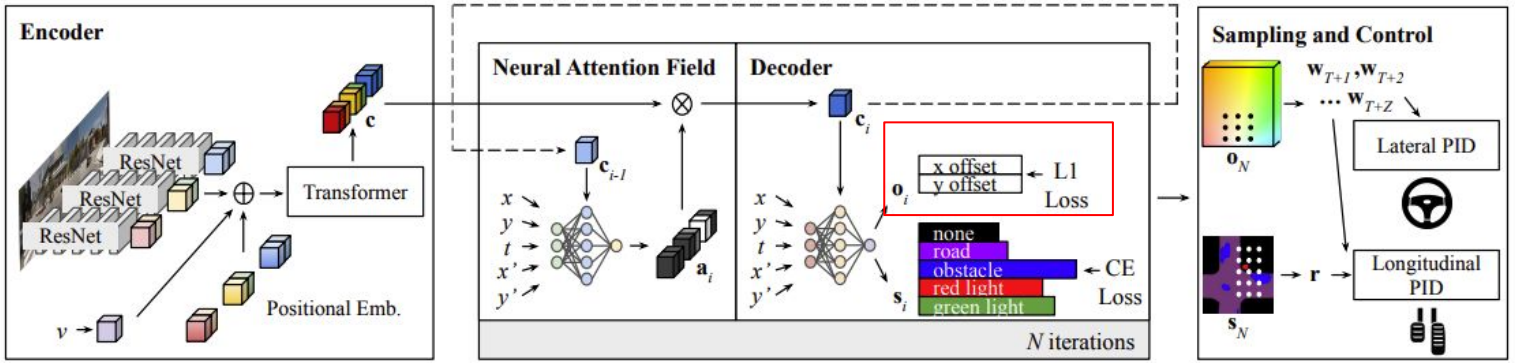
So waypoint is the model's control output

- The **encoder** is not informed by the query (x, y, t) or target (x', y') location
- Instead, **NEAT** identifies relevant patch features recurrently $(\dim(\mathbf{c}_i) \ll \dim(\mathbf{c}))$
- The **decoder** predicts the semantic class and waypoint offset from $\mathbf{c}_N$

* NEAT: Neural Attention Fields for End-to-End Autonomous Driving, ICCV 2021

End-to-end autonomous driving with BEV perception*

Architecture: Sampling and Control



- At test time, we decode **waypoints** and a **red light indicator** from our model
- To extract both, we uniformly sample a sparse grid and **query the decoder**
- Grid pooling provides robustness at inference time; control via **PID controllers**

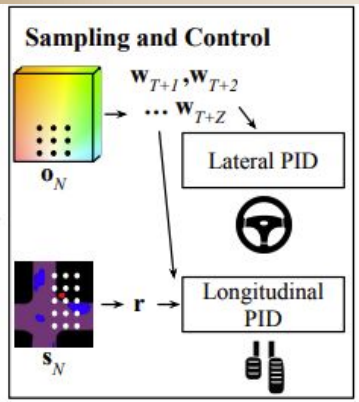
Converts the waypoint (x,y offsets) to steering and throttle/break command via PID

* NEAT: Neural Attention Fields for End-to-End Autonomous Driving, ICCV 2021

End-to-end autonomous driving with BEV perception*

- Given the predicted waypoints
- Make use of two PID controllers
  - One for steering (lateral PID)
  - One for throttle and brake (longitudinal)
    - Hard coded for red lights as well

You can take a closer look at the algorithm below from the paper. This is simple to implement.



Algorithm 2: Generating an action from waypoints.

input : $v, (x', y'), \mathbf{r}, \{\mathbf{w}_t\}_{t=T+1}^{T+Z}$; // velocity, red light indicator, target and waypoints

output: steer $\in (-1, 1)$, throttle $\in (0, 1)$, brake $\in (0, 1)$;

$\delta = 0$

for $t \leftarrow T+1$ **to** $T+Z-1$ **do**

  $\delta \mathrel{+}= \frac{1}{Z-1} ||\mathbf{w}_{t+1} - \mathbf{w}_t||$;

$\alpha^* = \tan^{-1}\left(\frac{(\mathbf{w}^*)[1]}{(\mathbf{w}^*)[0]}\right)$;

$\alpha_b = \tan^{-1}\left(\frac{y'}{x'}\right)$;

if $||\alpha^* - \alpha_b|| > \beta_{err}$ **or** $\alpha_b < \alpha^*$ **then**

  $\alpha = \alpha_b$

else

  $\alpha = \alpha^*$

steer $= \text{LatPID}(\alpha)$

if $\mathbf{r} = 1$ **or** $\delta < \beta_{min}$ **or** $\delta < v\beta_{ratio}$ **then**

  throttle $= 0$

  brake $= 1$

else

  throttle $= \text{LonPID}(\delta - v)$

  brake $= 0$

End-to-end autonomous driving with BEV perception*

- Given the predicted waypoints
- Make use of two PID controllers
  - One for steering (lateral PID)
  - One for throttle and brake (longitudinal)
    - Hard coded for red lights as well

Each controller has 3 gain parameters, which are tuned on a subset of training routes. For the speed controller, K_p = 5.0, K_i = 0.5, K_d = 1.0 and for the turn controller, K_p = 0.75, K_i = 0.75, K_d = 0.3

$$u = -\left(K_p e_{ct} + K_i \int e_{ct}(t)dt + K_d \dot{e}_{ct}\right)$$

Proportional (current) | Integral (past) | Derivative (future)

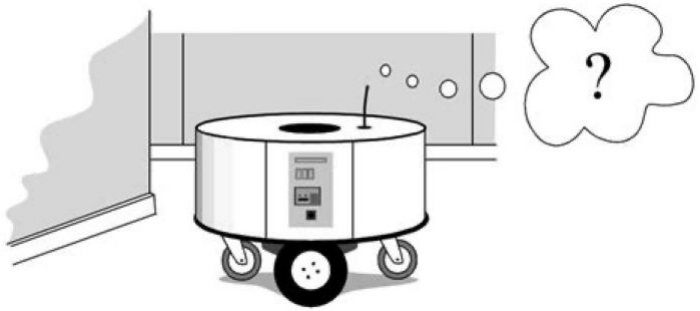
End-to-end autonomous driving with BEV perception*

- 

* NEAT: Neural Attention Fields for End-to-End Autonomous Driving, ICCV 2021

Agenda

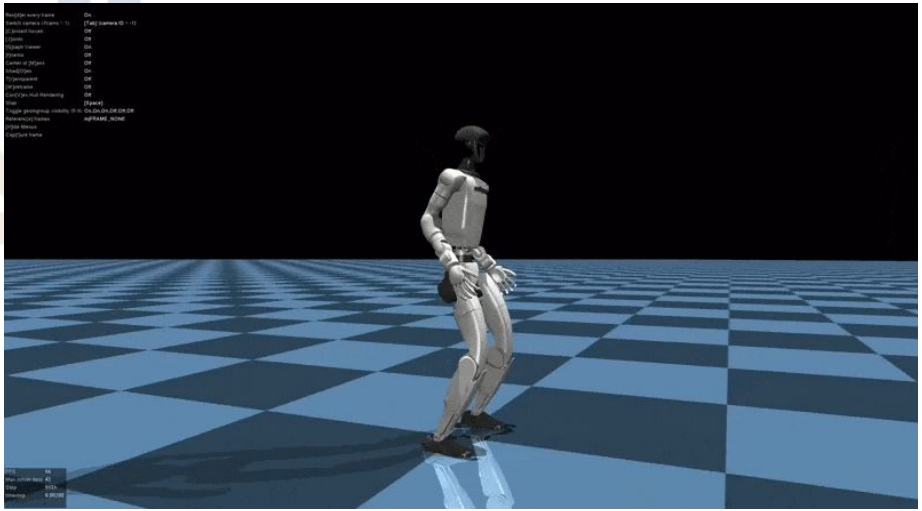
- Objectives
  - We have introduced classic state estimation methods. Now we will take a look at key classic robotics topics of planning and control. We will have Quiz test 2 and paper presentation #1.
- The tasks
  - Motion Planning
  - **Feedback Control**
    - Example for Self-driving Policy (Simulation)
    - **Example for Locomotion Policy (Real Robot)**
  - Quiz 2
  - Paper presentation #1

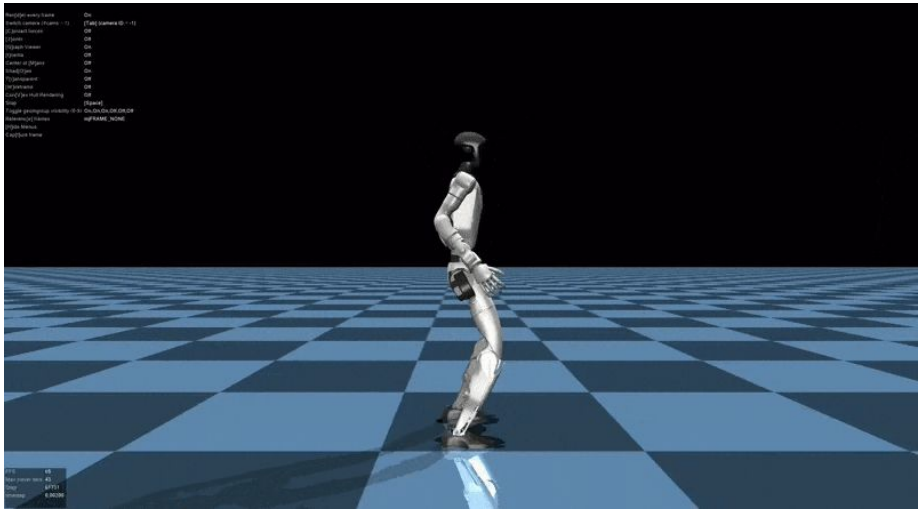


* Slide credit: Chris Mavrogiannis; Jeannette Bohg; Cyrill Stachniss; Hao Su

Deploying Locomotion Policy

We train a RL policy that can control the robot to walk and squat!





So given the observations and remote control command (base velocity and height), the policy outputs 12 actions (6 **joint position** for each leg). This is call “position control” 位置控制

Using Control in Real Robots

- When an RL work says: *we use “velocity control” or “position control” as action*. What does that mean?

Using Control in Real Robots

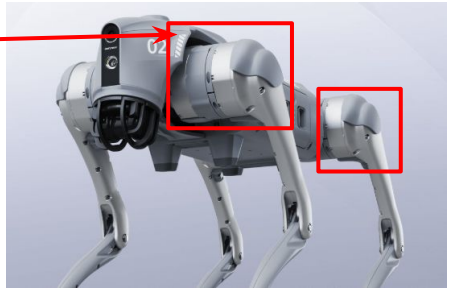
- It means the policy output (the robot action) can be “target joint velocity” or “target joint position” for a controller.

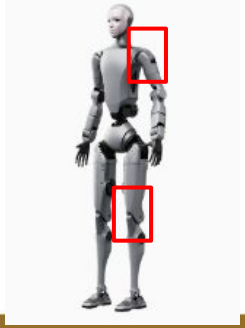
Using Control in Real Robots

- "target joint velocity" or "target joint position"
- A controller (such as PD) is used to convert this velocity or position signal to **joint torques**, which are then used to drive the robot.

Torques: rotational force. This is what the motor control wants: how much force == how much current/power to send to the motor







Most robot joints are rotational motors

Using Control in Real Robots

- “target joint velocity” or “target joint position”
- A controller (such as PD) is used to convert this velocity or position signal to **joint torques**, which are then used to drive the robot.
- **Joint velocity/position** may be a better choice for robot action (than torque) due to learnability and sim-to-real transferability.
    - So we rarely train a model to directly output torque

Using Control in Real Robots

- The motor of real robots uses torque control
- How do the real robots take **joint position control** and convert to **torque outputs**?
- It depends on the robot (check their manual!)
- Here is an example for Unitree G1
  - Manual: https://support.unitree.com/home/en/Motor_SDK_Dev_Guide/overview

Control in Real Robot - Unitree G1

"q" is commonly used to represent joint position

- How do the real robots take **joint position control** and convert to **torque outputs**? - G1 internally uses a PD controller

$$\tau = tau + kp * (q_{des} - q) + kd * (dq_{des} - dq)$$

The expected output torque of the motor, in units of N.m

Current motor position and speed

- Feedforward torque: tau — Often set to 0
- Desired angular position: q_{des} — This is what usually the RL policy outputs - joint position. Measured in rad/degree
- Desired angular velocity: dq_{des} — Sometimes you also want the joint to reach such position at a certain speed: measured in rad/s
- Position stiffness: kp
- Damping coefficient: kd

Stiffness and damping: very common concepts in robotics
Let's see some control mode to understand them

Control in Real Robot - Unitree G1

- How do the real robots take **joint position control** and convert to **torque outputs**? - G1 internally uses a PD controller

$$\tau = tau + kp * (q_{des} - q) + kd * (dq_{des} - dq)$$

- Feedforward torque: tau
- Desired angular position: q_{des}
- Desired angular velocity: dq_{des}
- Position stiffness: kp
- Damping coefficient: kd

Let's look at certain preset control mode:

- Position control mode
  - This is commonly used
  - Policy outputs joint position q
  - The PD controller computes the torque based on (q - q_current)
  - So if the joint is already at q, the torque output should be 0

```
cmd.q    = q
cmd.dq   = 0.0
cmd.kp   = 0.01
cmd.kd   = 0.01
cmd.tau  = 0.0
```

Normally, this cmd is send to the robot at >200Hz to ensure smooth motions

*https://support.unitree.com/home/en/Motor_SDK_Dev_Guide/control_mode

Control in Real Robot - Unitree G1

- How do the real robots take **joint position control** and convert to **torque outputs**? - G1 internally uses a PD controller

$$\tau = tau + kp * (q_{des} - q) + kd * (dq_{des} - dq)$$

- Feedforward torque: tau
- Desired angular position: q_{des}
- Desired angular velocity: dq_{des}
- Position stiffness: kp
- Damping coefficient: kd

Let's look at certain preset control mode:

- Damping mode
  - This is often used to safely stop the robot
  - We want all the joint to stop!!
  - If the current joint speed is high, an impedance torque will generate to slow it down
  - The output torque will be zero when joint speed is zero

```
cmd.q = 0.0;
cmd.dq = 0.0;
cmd.kp = 0.0;
cmd.kd = 2.0;
cmd.tau = 0.0;
```


*https://support.unitree.com/home/en/Motor_SDK_Dev_Guide/control_mode

Control in Real Robot - Unitree G1

- How do the real robots take **joint position control** and convert to **torque outputs**? - G1 internally uses a PD controller

$$\tau = tau + kp * (q_{des} - q) + kd * (dq_{des} - dq)$$

- Feedforward torque: tau
- Desired angular position: q_{des}
- Desired angular velocity: dq_{des}
- Position stiffness: kp
- Damping coefficient: kd

Let's look at certain preset control mode:

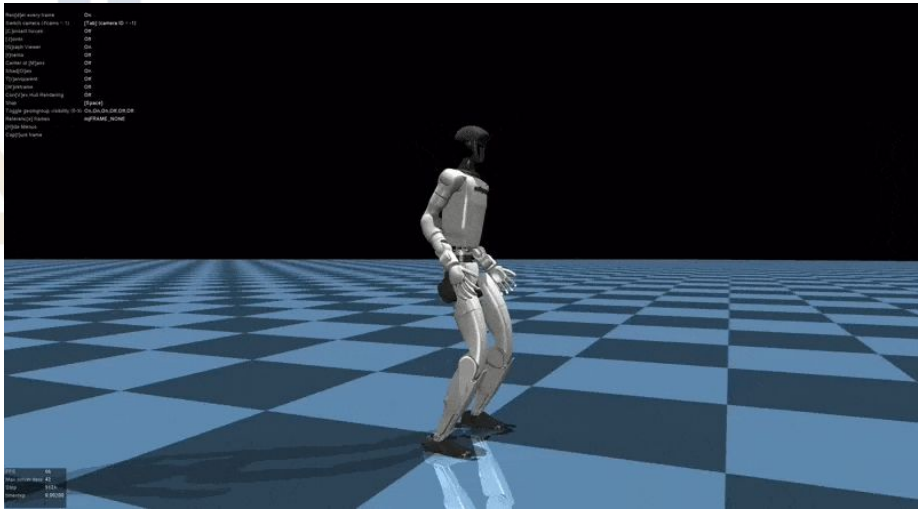
- Zero-torque mode
  - The joint outputs zero torque like it is without power
  - As opposed to the damping mode
  - We normally do not use zero-torque mode to emergency stop
    - So the robot can also be protected
    - Why?

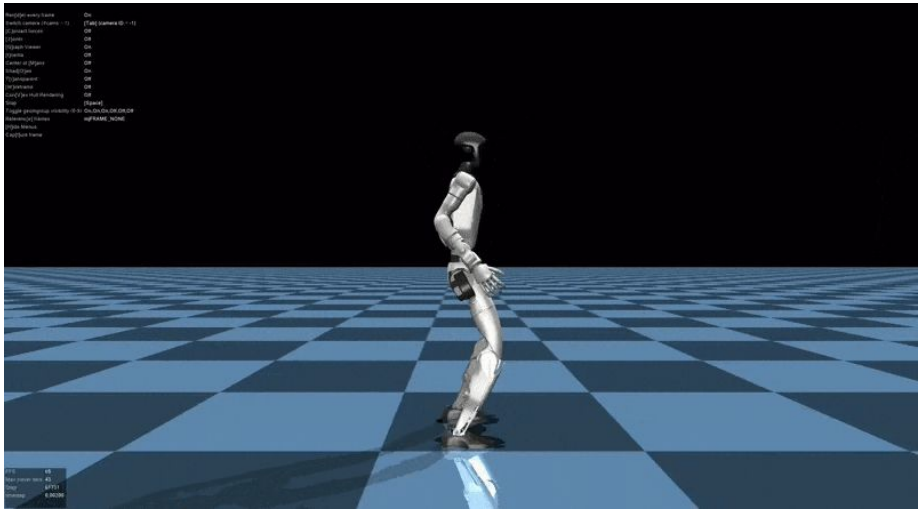
```
cmd.q = 0.0;
cmd.dq = 0.0;
cmd.kp = 0.0;
cmd.kd = 0.0;
cmd.tau = 0.0;
```


*https://support.unitree.com/home/en/Motor_SDK_Dev_Guide/control_mode

Deploying Locomotion Policy

We train a RL policy that can control the robot to walk and squat!





So given the observations and remote control command (base velocity and height), the policy outputs 12 actions (6 **joint position** for each leg). So we have the **q_des** at 200Hz

$$\tau = tau + kp * (q_{des} - q) + kd * (dq_{des} - dq)$$

Deploying Locomotion Policy

$$\tau = tau + kp * (q_{des} - q) + kd * (dq_{des} - dq)$$

We train a RL policy that can control the robot to walk and squat!



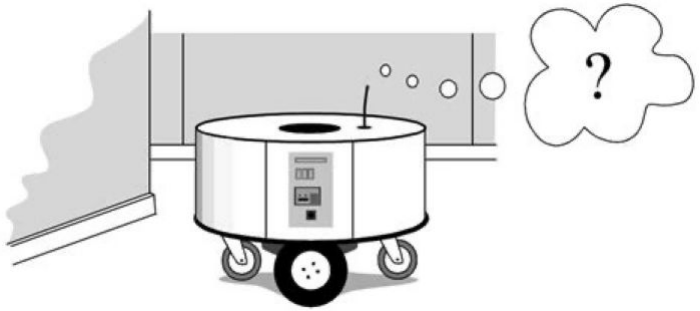
So what about the gain parameters, Kp and Kd? This needs to be tuned in real robot….

From this example, you can see the robot's motion is a bit stiff. In this case, we may need to reduce Kp first.

Kp acts like spring stiffness and Kd like damping

Agenda

- Objectives
  - We have introduced classic state estimation methods. Now we will take a look at key classic robotics topics of planning and control. We will have Quiz test 2 and paper presentation #1.
- The tasks
  - Motion Planning
  - Feedback Control
    - Example for Self-driving Policy (Simulation)
    - Example for Locomotion Policy (Real Robot)
  - Quiz 2
  - Paper presentation #1



* Slide credit: Chris Mavrogiannis; Jeannette Bohg; Cyrill Stachniss; Hao Su

Quiz 2

- https://hkust-gz.instructure.com/courses/3918/assignments/29203
  - Access Code
  -
-

Paper Presentation #1 - 10/12

- Aim for about 9 minutes; hard stop at 10 minutes
- The teacher and TA will grade you based on the rubrics

Presentation Date	Time	Name	Paper #	Theme	Paper Title
2026.10.12	10:20–10:35	Yuming ZHANG	23	Navigation & Social Navigation	ViNT: A Foundation Model for Visual Navigation
2026.10.12	10:35–10:50	Yilin WU	15	Robotic Manipulation & Affordance	RVT: Robotic View Transformer for 3D Object Manipulation
2026.10.12	10:50–11:05	Weihao LIU	37	3D & Egocentric Perception	3EED: Ground Everything Everywhere in 3D
2026.10.12	11:05–11:20	Fanxi JING	3	Robot Foundation Models / VLA	Exploring the Limits of Vision-Language-Action Manipulations in Cross-task Generalization
2026.10.12	11:20–11:35	Xuqiao MA	39	3D & Egocentric Perception	Open-Vocabulary 3D Semantic Segmentation with Text-to-Image Diffusion Models
2026.10.12	11:35–11:50	Chenhui LIU	2	Robot Foundation Models / VLA	DiT4DiT: Jointly Modeling Video Dynamics and Actions for Generalizable Robot Control

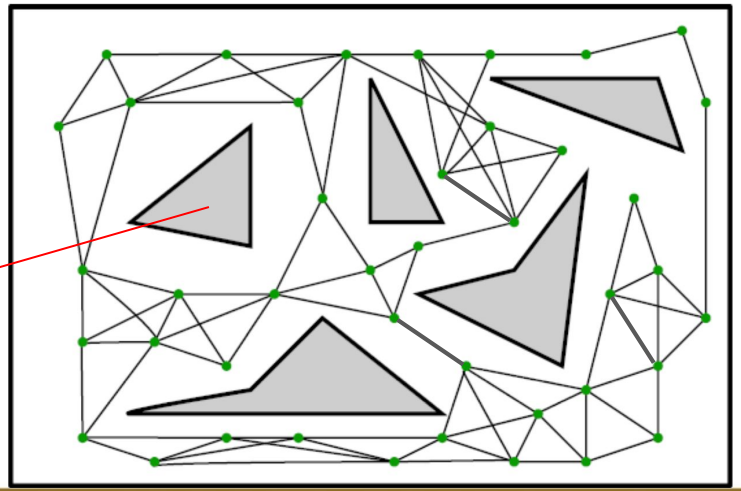
Backup Slides on Sampling-based Motion Planning
(These won’t be included in the quiz)

Sampling-based Motion Planning - PRM Example

- Phase 1: Map construction
  - Randomly sample collision-free configurations
  - Connect every sampled state to its neighbors
  - Connect the start and goal states to the **graph**

This is a 2D example.
The robot could be any shape.

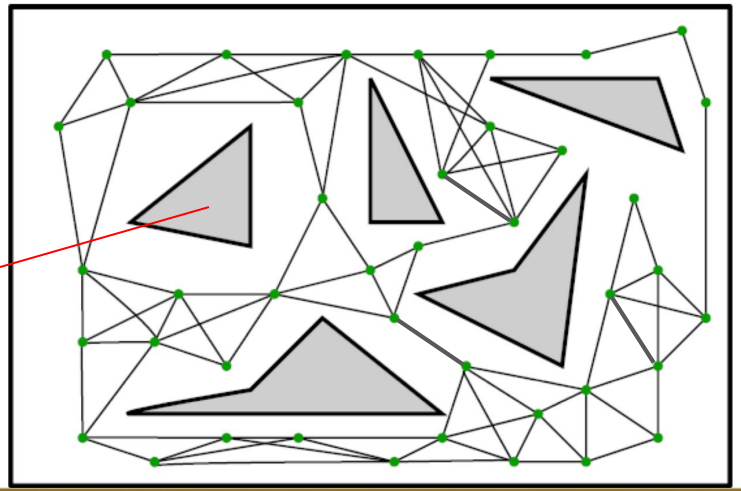
Obstacles



Sampling-based Motion Planning - PRM Example

- Phase 1: Map construction
  - Randomly sample collision-free configurations
  - Connect every sampled state to its neighbors
  - Connect the start and goal states to the **graph**

This is a 2D example.
The robot could be any shape.



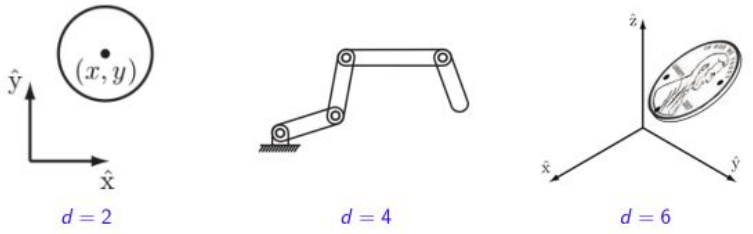
Obstacles

What is collision-free configurations?

Configuration Space (C-Space)

- Captures all degrees of freedom (all rigid body transformations/placements)
- For example, consider a polygonal robot (e.g., a triangle) in 2D plane $\mathcal{R} \subset \mathbb{R}^2$
- The robot can rotate by angle θ or translate $(x_t, y_t) \subset \mathbb{R}^2$ So 3 DoF
- **Every combination $q = (x_t, y_t, \theta)$ yields a unique robot placement: configuration**
- So, C-space is **a subset of** $\mathbb{R}^3$
- Note: $\theta \pm 2\pi$ yields equivalent rotations ⇒ C-space is: $\mathbb{R}^2 \times \mathcal{S}^1$
  - Also called SE(2): "**Special Euclidean group** of rigid body displacements in two-dimensions"
- Concept of C-space extends naturally to higher dimensions (e.g., robot linkages)

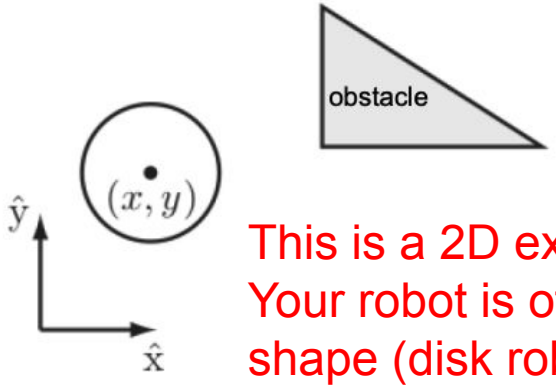
S^1: 1-dimensional sphere



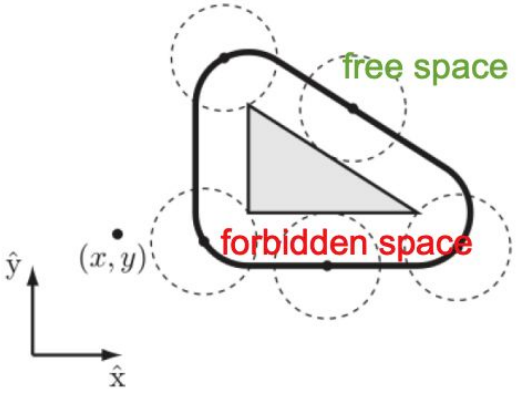
SE(3) for 3D space: Usually a 4x4 matrix to represent a pose (3x3 rotation + 3x1 translation)
Position + Orientation

Free-space Computation

- The free space is not known in advance
- We need to compute this space given the ingredients
  - Robot representation, i.e., its shape (polygon, polyhedron, …)
  - Representation of obstacles
- To achieve this, we do the following:
  - Contract the robot into a point
  - In return, inflate (or stretch) obstacles by the shape of the robots

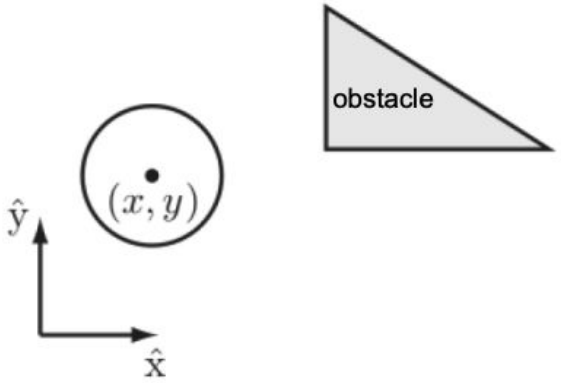


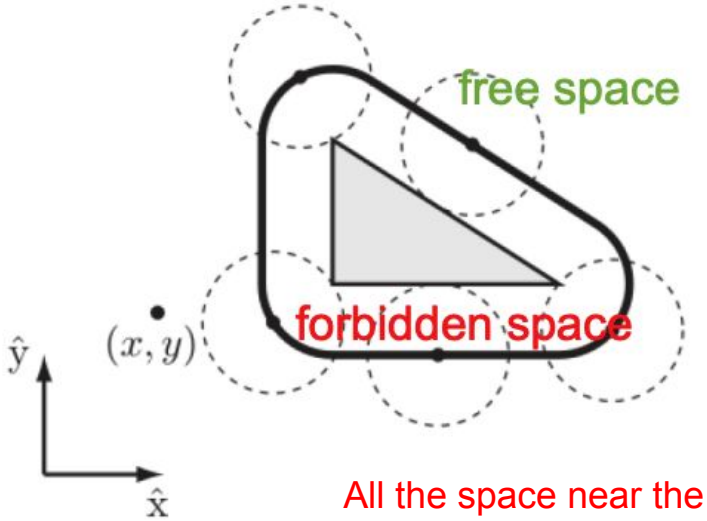
This is a 2D example. Your robot is of round shape (disk robot)



Configuration free space

- Consider a disk robot and the obstacle
  - The subset of **all collision free configurations** is the free space





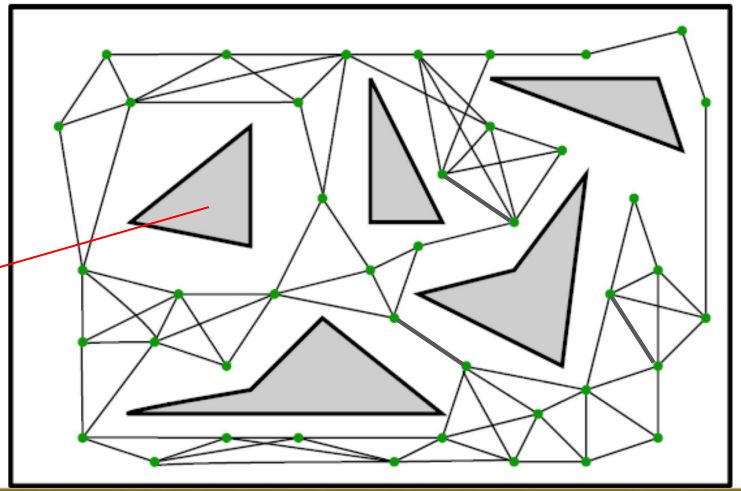
All the space near the obstacle within is obstacle space

Sampling-based Motion Planning - PRM Example

- Phase 1: Map construction
  - Randomly sample collision-free configurations
  - Connect every sampled state to its neighbors
  - Connect the start and goal states to the **graph**

This is a 2D example.
The robot could be any shape.

Obstacles



Sampling-based Motion Planning - PRM Example

- Phase 2: Query
  - Run path finding algorithms like Dijkstra or A*

